DTU Compute
Department of Applied Mathematics and Computer Science

Internship Report

Stochastic Bilinear MPC with Chance Constraints on District Heating Systems

Taha Berk Koç

01.07.2026 - 27.09.2026

DTU Compute
Department of Applied Mathematics and Computer Science
Technical University of Denmark

Matematiktorvet
Building 303B
2800 Kongens Lyngby, Denmark
Phone +45 4525 3031
compute@compute.dtu.dk
www.compute.dtu.dk

Summary

This report summarizes the work carried out during the internship on the development of Model Predictive Control (MPC) methods for district heating systems under uncertainty. The study progresses from linear time-invariant models to bilinear and stochastic formulations, including chance constraints, state-input analysis, and Kalman filtering. These components are then combined in a stochastic bilinear MPC framework and applied to a single-pipe district heating system. Both open-loop and closed-loop implementations are evaluated using Monte Carlo simulations. The final framework considers uncertainty, state estimation, probabilistic constraints, and nonlinear optimization within a unified control approach.

Acknowledgements

I would like to express my sincere gratitude to Assoc. Prof. Tobias K. S. Ritschel for his guidance, support, and encouragement throughout my research at the Technical University of Denmark. His valuable insights, constructive feedback, and thoughtful discussions have greatly contributed to my understanding of model predictive control and helped me develop as a researcher. I am especially thankful for the opportunity to work under his supervision and for the trust and independence he has given me throughout this work.

Contents

Summary i

Acknowledgements ii

Contents iii

1 Introduction 1

2 Linear Time-Invariant Model 2
- 2.1 LTI System . . . 2
- 2.2 Optimal Control Problem . . . 5
- 2.3 Open-Loop Implementation . . . 6
- 2.4 Closed-Loop Implementation . . . 12

3 Bilinear Model 17
- 3.1 Bilinear System . . . 17
- 3.2 Optimal Control Problem . . . 19
- 3.3 Open-Loop Implementation . . . 21
- 3.4 Closed-Loop Implementation . . . 23

4 Stochastic Model 25
- 4.1 Stochastic System . . . 25
- 4.2 Model with Chance Constraints . . . 29
- 4.3 Stacked Forms Derivation . . . 31
- 4.4 Open-Loop Implementation . . . 36
- 4.5 Closed-Loop Implementation . . . 40

5 Stochastic Bilinear Model with Chance Constraints 42
- 5.1 Stochastic Bilinear System . . . 42
- 5.2 Model with Chance Constraints . . . 46
- 5.3 Cost Function Derivation . . . 47
- 5.4 Problem Overview . . . 48
- 5.5 Open-Loop Implementation . . . 48
- 5.6 Closed-Loop Implementation . . . 53

6 State Input Analysis **55**
6.1 Analysing Objective 55
6.2 Results 56

7 Kalman Filtering **59**
7.1 Theory Derivation 59
7.2 Results 62

8 District Heating System **69**
8.1 Derivation of $\dot{T}_i$ Equation 69
8.2 Change of Variable 71
8.3 Controller and Hidden Plant Approach 73
8.4 Updates on Implementation 76
8.5 Results 78

9 Conclusion **80**

A An Appendix **81**

CHAPTER 1

Introduction

District heating systems require effective control strategies to maintain desired temperature levels while dealing with nonlinear dynamics, disturbances, and uncertainty. Model Predictive Control provides a suitable framework for this purpose by predicting future system behavior and optimizing control actions over a finite horizon. The aim of this report was to progressively develop an MPC framework, starting from deterministic linear models and extending it to bilinear and stochastic systems with chance constraints. State estimation using Kalman filtering was also investigated. Finally, the developed methods were applied to a district heating pipe model to study their behavior in a more realistic control problem.

CHAPTER 2

Linear Time-Invariant Model

2.1 LTI System

A discrete-time Linear Time-Invariant (LTI) system can be represented in state-space form as

$$x_{k+1} = Ax_k + Bu_k, \tag{2.1}$$
$$y_k = Cx_k, \tag{2.2}$$

where $x_k \in R^n$ denotes the state vector, $u_k \in R^m$ denotes the input vector, and $y_k \in R^r$ denotes the output vector at time step k. The matrices A, B, and C are the state, input, and output matrices, respectively.

For model predictive control, the future behavior of the system is predicted from the current state x_0 and a sequence of future control inputs

$$u_0, u_1, \ldots, u_{N-1}, \tag{2.3}$$

where N denotes the prediction horizon.

2.1.1 State and Output Prediction

Starting from the initial state x_0, the state after one time step is obtained from (2.1) as

$$x_1 = Ax_0 + Bu_0 \tag{2.4}$$

At the next time step,

$$x_2 = Ax_1 + Bu_1 \tag{2.5}$$
$$= A(Ax_0 + Bu_0) + Bu_1 \tag{2.6}$$
$$= A^2x_0 + ABu_0 + Bu_1 \tag{2.7}$$

Similarly, the third predicted state is

$$x_3 = Ax_2 + Bu_2 \tag{2.8}$$
$$= A^3x_0 + A^2Bu_0 + ABu_1 + Bu_2 \tag{2.9}$$

Following the same recursion, the state at an arbitrary prediction step k can be written as

$$x_k = A^k x_0 + \sum_{i=0}^{k-1} A^{k-1-i} B u_i \tag{2.10}$$

Equation (2.10) separates the future state into two contributions. The first term represents the evolution of the initial state in the absence of future inputs, whereas the summation represents the contribution of the input sequence.

Using the output relation in (2.2), the corresponding output at prediction step k is

$$y_k = Cx_k \tag{2.11}$$
$$= C\left(A^k x_0 + \sum_{i=0}^{k-1} A^{k-1-i} B u_i\right), \tag{2.12}$$

which gives

$$y_k = CA^k x_0 + \sum_{i=0}^{k-1} CA^{k-1-i} B u_i \tag{2.13}$$

Thus, each predicted output is expressed as a function of the current state and all control inputs applied before that prediction step.

2.1.2 Stacked Prediction Formulation

For optimization over a prediction horizon N, the predicted outputs and future inputs can be collected into stacked vectors. The output vector is defined as

$$Y = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_N \end{bmatrix} \in R^{Nr}, \tag{2.14}$$

and the corresponding input vector is

$$U = \begin{bmatrix} u_0 \\ u_1 \\ \vdots \\ u_{N-1} \end{bmatrix} \in R^{Nm} \tag{2.15}$$

The first two predicted outputs can be written explicitly as

$$y_1 = CAx_0 + CBu_0, \tag{2.16}$$
$$y_2 = CA^2x_0 + CABu_0 + CBu_1, \tag{2.17}$$

and the final output over the prediction horizon is

$$y_N = CA^Nx_0 + CA^{N-1}Bu_0 + CA^{N-2}Bu_1 + \cdots + CBu_{N-1} \tag{2.18}$$

Stacking these equations gives

$$\begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_N \end{bmatrix} = \begin{bmatrix} CA \\ CA^2 \\ \vdots \\ CA^N \end{bmatrix} x_0 + \begin{bmatrix} CB & 0 & \cdots & 0 \\ CAB & CB & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ CA^{N-1}B & CA^{N-2}B & \cdots & CB \end{bmatrix} \begin{bmatrix} u_0 \\ u_1 \\ \vdots \\ u_{N-1} \end{bmatrix} \tag{2.19}$$

Introducing the prediction matrices

$$\Phi = \begin{bmatrix} CA \\ CA^2 \\ \vdots \\ CA^N \end{bmatrix} \in R^{Nr \times n}, \tag{2.20}$$

and

$$\Gamma = \begin{bmatrix} CB & 0 & \cdots & 0 \\ CAB & CB & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ CA^{N-1}B & CA^{N-2}B & \cdots & CB \end{bmatrix} \in R^{Nr \times Nm}, \tag{2.21}$$

the stacked prediction equation can be written compactly as

$$\boxed{Y = \Phi x_0 + \Gamma U} \tag{2.22}$$

The term

$$Y_{\text{free}} = \Phi x_0 \tag{2.23}$$

describes the *free response* of the system, i.e. the predicted output evolution due only to the current state. The term

$$Y_{\text{forced}} = \Gamma U \tag{2.24}$$

describes the *forced response* resulting from the future input sequence. Consequently,

$$Y = Y_{\text{free}} + Y_{\text{forced}} = \Phi x_0 + \Gamma U \tag{2.25}$$

2.2 Optimal Control Problem

After deriving the stacked prediction model, the next step is to determine the sequence of control inputs that minimizes a desired performance criterion over the prediction horizon.

For a prediction horizon of length N, the quadratic cost function is defined as

$$\phi = \sum_{k=1}^{N} (y_k - \bar{y}_k)^T Q_k (y_k - \bar{y}_k) + \sum_{k=0}^{N-1} u_k^T R_k u_k, \tag{2.26}$$

where $\bar{y}_k$ denotes the desired reference output, while $Q_k \succeq 0$ and $R_k \succ 0$ are weighting matrices penalizing the output tracking error and the control effort, respectively.

Using the stacked output and input vectors, the cost function can be written as

$$\phi = (Y - \bar{Y})^T Q (Y - \bar{Y}) + U^T R U \tag{2.27}$$

From the stacked prediction equation

$$Y = \Phi x_0 + \Gamma U, \tag{2.28}$$

the cost function becomes

$$\phi = (\Phi x_0 + \Gamma U - \bar{Y})^T Q (\Phi x_0 + \Gamma U - \bar{Y}) + U^T R U \tag{2.29}$$

Defining

$$e = \Phi x_0 - \bar{Y}, \tag{2.30}$$

the cost function can be written as

$$\phi = (e + \Gamma U)^T Q (e + \Gamma U) + U^T R U \tag{2.31}$$

Expanding the quadratic term gives

$$\phi = e^T Q e + e^T Q \Gamma U + U^T \Gamma^T Q e + U^T \Gamma^T Q \Gamma U + U^T R U \tag{2.32}$$

Since the cross terms are scalar and $Q = Q^T$,

$$e^T Q \Gamma U = U^T \Gamma^T Q e \tag{2.33}$$

Therefore,

$$\phi = e^T Q e + 2U^T \Gamma^T Q e + U^T (\Gamma^T Q \Gamma + R) U \tag{2.34}$$

To obtain the optimal control sequence, the cost function is differentiated with respect to U:

$$\frac{\partial \phi}{\partial U} = 2\Gamma^T Q e + 2(\Gamma^T Q \Gamma + R) U \tag{2.35}$$

At the optimum,

$$\frac{\partial \phi}{\partial U} = 0 \tag{2.36}$$

Hence,

$$\Gamma^T Q (\Phi x_0 - \bar{Y}) + (\Gamma^T Q \Gamma + R) U = 0 \tag{2.37}$$

Solving for U gives the optimal control sequence

$$\boxed{U^* = -(\Gamma^T Q \Gamma + R)^{-1} \Gamma^T Q (\Phi x_0 - \bar{Y})} \tag{2.38}$$

Equivalently,

$$\boxed{U^* = (\Gamma^T Q \Gamma + R)^{-1} \Gamma^T Q (\bar{Y} - \Phi x_0)} \tag{2.39}$$

2.3 Open-Loop Implementation

After deriving the optimal control sequence for the LTI system, the controller was first implemented in an open-loop setting. In this approach, the complete optimal input sequence is computed once using the initial state and the desired reference trajectory. The resulting inputs are then applied sequentially to the system without recalculating the optimization problem using updated state information.

The implementation procedure can be summarized in the following steps.

2.3.1 Implementation Procedure

First, the prediction horizon N, the reference trajectory $\bar{Y}$, the weighting matrices Q and R, and the initial state x_0 are specified. The weighting matrices satisfy

$$Q \succeq 0, \qquad R \succ 0 \tag{2.40}$$

The stacked prediction matrices Φ and Γ are then constructed according to

$$Y = \Phi x_0 + \Gamma U, \tag{2.41}$$

where

$$\Phi = \begin{bmatrix} CA \\ CA^2 \\ \vdots \\ CA^N \end{bmatrix}, \tag{2.42}$$

and

$$\Gamma = \begin{bmatrix} CB & 0 & \cdots & 0 \\ CAB & CB & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ CA^{N-1}B & CA^{N-2}B & \cdots & CB \end{bmatrix} \tag{2.43}$$

Using these matrices, the quadratic cost function is written as

$$\phi(U) = (\Phi x_0 + \Gamma U - \bar{Y})^T Q (\Phi x_0 + \Gamma U - \bar{Y}) + U^T R U \tag{2.44}$$

As derived in the previous section, minimizing this cost function with respect to U gives the optimal input sequence

$$U^* = -\left(\Gamma^T Q \Gamma + R\right)^{-1} \Gamma^T Q \left(\Phi x_0 - \bar{Y}\right) \tag{2.45}$$

In the open-loop implementation, (2.45) is evaluated only once. The resulting control sequence

$$U^* = \begin{bmatrix} u_0^* & u_1^* & \cdots & u_{N-1}^* \end{bmatrix}^T \tag{2.46}$$

is subsequently applied to the system according to

$$x_{k+1} = Ax_k + Bu_k^*, \tag{2.47}$$

$$y_k = Cx_k \tag{2.48}$$

Therefore, no new optimization problem is solved after the simulation begins. This distinguishes the open-loop implementation from a receding-horizon or Model

Predictive Control implementation, in which the optimization problem is repeatedly solved using updated state information.

The complete procedure can therefore be summarized as

1. Select N, $\bar{Y}$, Q, R, and x_0.
2. Construct the prediction matrices Φ and Γ.
3. Construct the quadratic cost function $\phi(U)$.
4. Calculate the optimal input sequence U^*.
5. Apply the components of U^* sequentially to the system.
6. Compare the resulting output trajectory with the reference trajectory.

To quantify the tracking performance, the tracking error is defined as

$$E = Y - Y_{\text{ref}} \tag{2.49}$$

The Root Mean Square Error (RMSE) can additionally be used as a scalar measure of the tracking accuracy. For a system with r outputs, it is given by

$$\text{RMSE} = \sqrt{\frac{1}{Nr} \sum_{k=1}^{N} \sum_{j=1}^{r} \left(Y_{k,j} - Y_{\text{ref},k,j}\right)^2} \tag{2.50}$$

2.3.2 Results

The open-loop controller was evaluated using the following discrete-time LTI system:

$$A = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}, \qquad B = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \qquad C = \begin{bmatrix} 1 & 0 \end{bmatrix} \tag{2.51}$$

The desired output reference was changed every ten simulation steps. Three reference levels were considered:

$$\bar{y} = \begin{bmatrix} 1.0 & 3.0 & 5.0 \end{bmatrix} \tag{2.52}$$

To investigate the effect of the input penalty on the controller behavior, three different values of R were tested while keeping the remaining controller parameters unchanged:

$$R = 0.1, \qquad R = 0.5, \qquad R = 5 \tag{2.53}$$

Case 1: $R = 0.1$

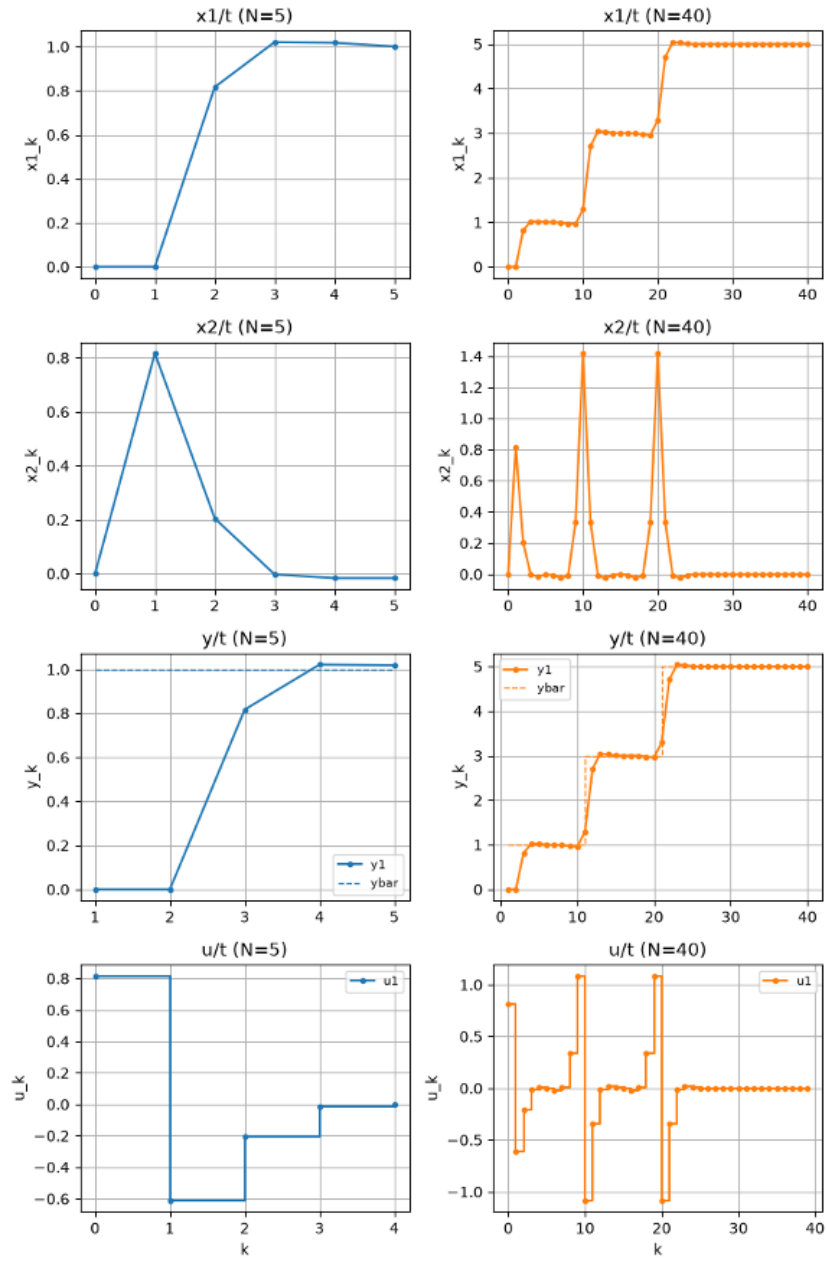


Figure 2.1: Open-loop tracking performance for $R = 0.1$, Left-side is when N=5 with the averaged RMSE 0.450392, right-side is the case when N=40 with the averaged RMSE 0.172099.

Case 2: $R = 0.5$

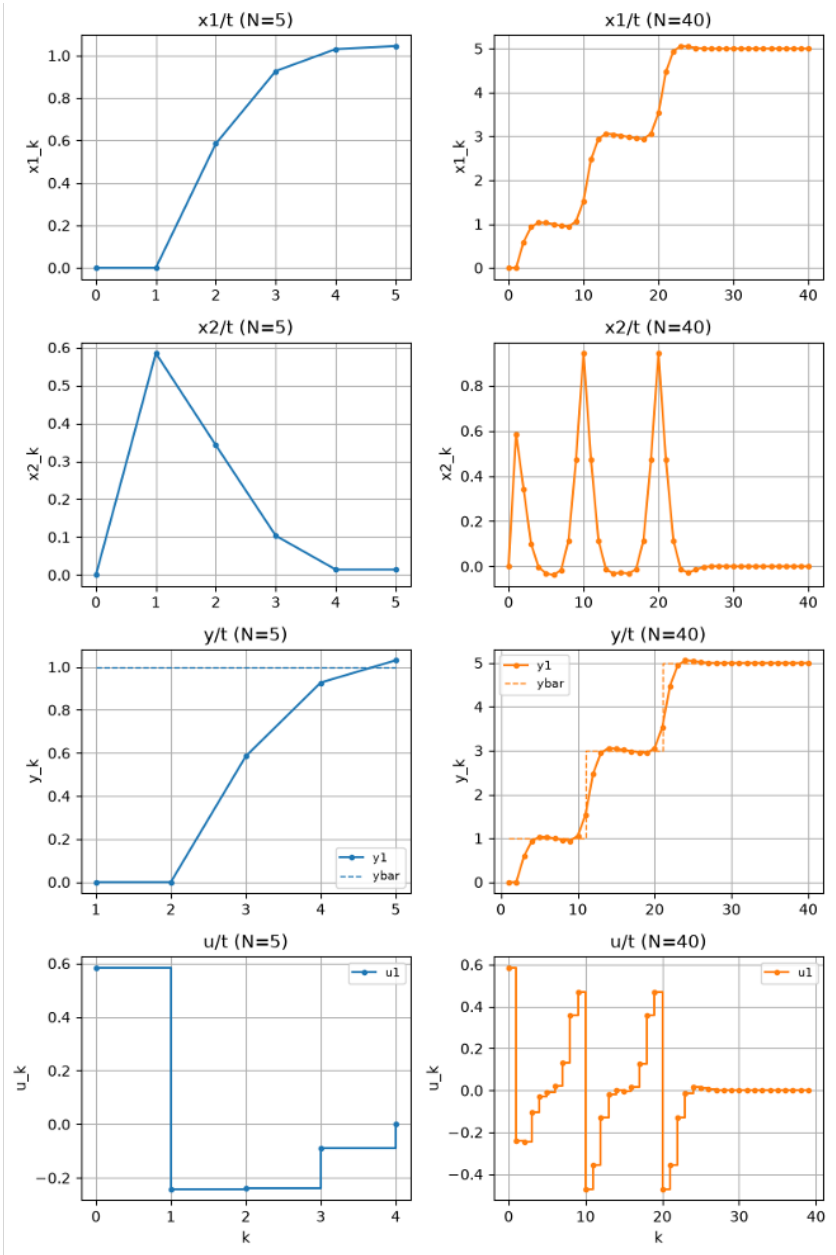


Figure 2.2: Open-loop tracking performance for $R = 0.5$, Left-side is when N=5 with the averaged RMSE 0.469931, right-side is the case when N=40 with the averaged RMSE 0.220377.

Case 3: $R = 5.0$

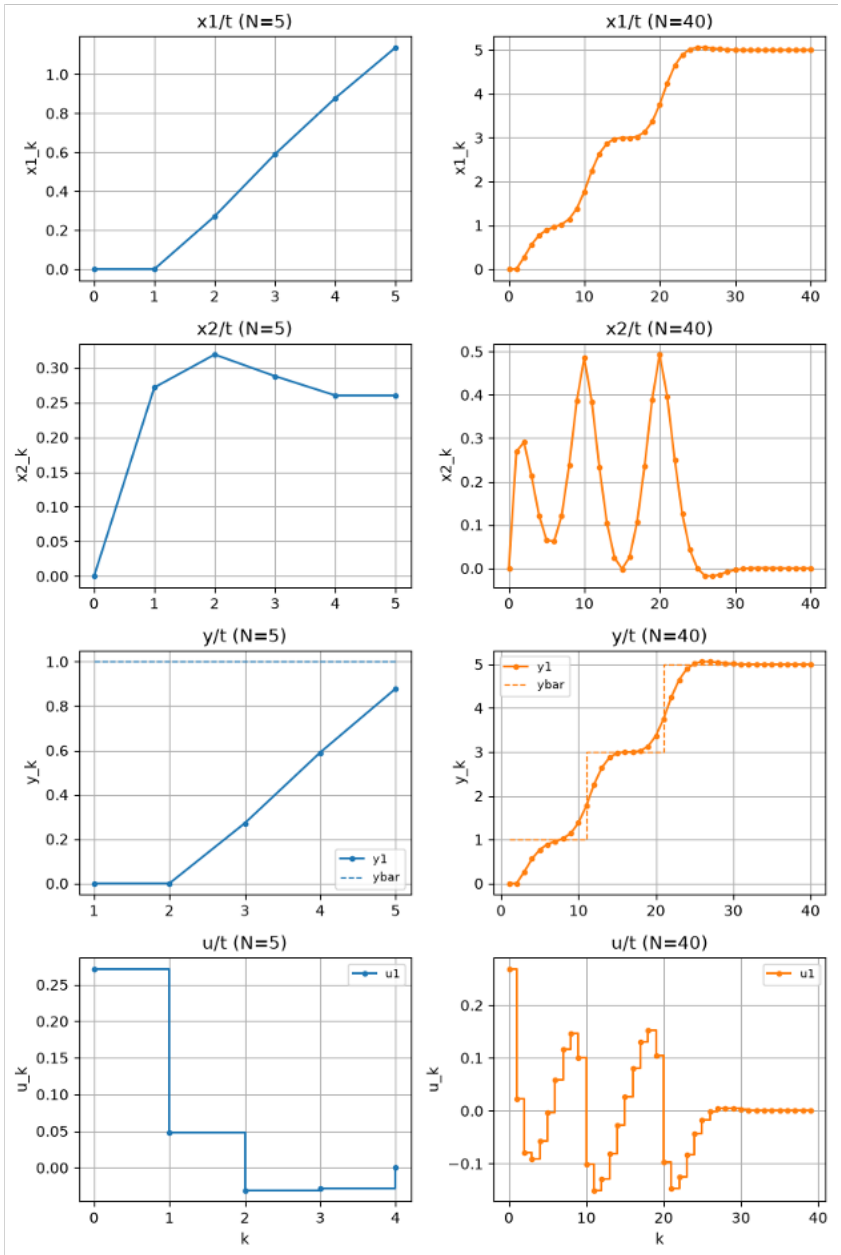


Figure 2.3: Open-loop tracking performance for $R = 5.0$, Left-side is when N=5 with the averaged RMSE 0.571764, right-side is the case when N=40 with the averaged RMSE 0.344596.

Comparison

The results demonstrate the trade-off between reference tracking performance and control effort introduced by the weighting matrix R. As the value of R increases, the contribution of the term $U^T RU$ to the objective function becomes more significant. Consequently, the optimizer increasingly penalizes large control inputs and attempts to reduce the magnitude of U.

For a small value such as $R = 0.1$, the controller is willing to use relatively large control actions in order to achieve accurate reference tracking. As R is increased to 0.5 and subsequently to 5, the control actions become less aggressive. The system therefore has increasing difficulty following changes in the reference trajectory, resulting in larger tracking errors.

Hence, increasing R improves the economy of the control action at the expense of tracking performance. The selection of R therefore represents a compromise between minimizing the control effort and maintaining accurate reference tracking.

2.4 Closed-Loop Implementation

The open-loop implementation computes the optimal control sequence only once from the initial state and applies the complete sequence without further optimization. In contrast, the closed-loop implementation repeatedly solves the optimal control problem using the most recent state information.

This approach follows the receding-horizon principle. At each time step, a new optimal input sequence is calculated, but only its first control action is applied to the system. At the following time step, the state is updated and the optimization problem is solved again.

2.4.1 Implementation Procedure

The prediction horizon N, control horizon N_c, total simulation length N_{end}, reference trajectory, weighting matrices Q and R, and initial state x_0 are first specified.

The prediction and control horizons satisfy

$$N_c \leq N \tag{2.54}$$

The stacked prediction model derived previously is

$$Y = \Phi x_k + \Gamma U_{\text{full}}, \tag{2.55}$$

where

$$\Phi = \begin{bmatrix} CA \\ CA^2 \\ \vdots \\ CA^N \end{bmatrix} \tag{2.56}$$

and

$$\Gamma = \begin{bmatrix} CB & 0 & \cdots & 0 \\ CAB & CB & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ CA^{N-1}B & CA^{N-2}B & \cdots & CB \end{bmatrix} \tag{2.57}$$

When a control horizon N_c shorter than the prediction horizon is used, the independent control variables are collected in

$$U_c = \begin{bmatrix} u_0 \\ u_1 \\ \vdots \\ u_{N_c-1} \end{bmatrix} \tag{2.58}$$

The complete input sequence over the prediction horizon is then generated as

$$U_{\text{full}} = TU_c, \tag{2.59}$$

where T is a control-horizon mapping matrix. This matrix maps the N_c independent control variables to the full prediction horizon.

Substituting (2.59) into the prediction model gives

$$Y = \Phi x_k + \Gamma T U_c \tag{2.60}$$

At each time step k, the reference trajectory corresponding to the current prediction window is denoted by $\bar{Y}_k$. The quadratic cost function is therefore written as

$$\phi(U_c) = \left(\Phi x_k + \Gamma T U_c - \bar{Y}_k\right)^T Q \left(\Phi x_k + \Gamma T U_c - \bar{Y}_k\right) + U_c^T R U_c \tag{2.61}$$

The optimization problem is solved again at every sampling instant. Following the same derivation as in the previous section, the optimal control sequence becomes

$$U_c^* = -\left[(\Gamma T)^T Q (\Gamma T) + R\right]^{-1} (\Gamma T)^T Q \left(\Phi x_k - \bar{Y}_k\right) \tag{2.62}$$

Unlike the open-loop implementation, the complete optimal sequence is not applied to the system. Instead, only the first control input is selected.

A selection matrix Π can be defined such that

$$u_k = \Pi U_c^* \tag{2.63}$$

For a scalar-input system,

$$\Pi = \begin{bmatrix} 1 & 0 & \cdots & 0 \end{bmatrix} \tag{2.64}$$

The selected input is then applied to the system:

$$x_{k+1} = Ax_k + Bu_k, \tag{2.65}$$
$$y_k = Cx_k \tag{2.66}$$

After obtaining the new state x_{k+1}, the prediction window is shifted forward by one time step. The reference trajectory is updated accordingly, and the optimization problem is solved again.

This procedure is repeated until

$$k = N_{\text{end}} - 1 \tag{2.67}$$

Therefore, the closed-loop algorithm can be summarized as follows:

1. Select N, N_c, N_{end}, Q, R, the reference trajectory, and x_0.
2. Construct the prediction matrices Φ and Γ, together with the control-horizon mapping matrix T.
3. At time step k, construct the reference vector $\bar{Y}_k$ over the current prediction horizon.
4. Compute the optimal control sequence U_c^*.
5. Apply only the first control action,

$$u_k = \Pi U_c^*$$

6. Update the system state and output.
7. Shift the prediction horizon and repeat the optimization for $k + 1$.

2.4.2 Results

The closed-loop controller was evaluated using the same LTI system considered in the open-loop implementation:

$$A = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}, \qquad B = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \qquad C = \begin{bmatrix} 1 & 0 \end{bmatrix} \tag{2.68}$$

The reference output was chosen as

$$\bar{y} = \begin{bmatrix} 1.0 & 3.0 & 5.0 \end{bmatrix}, \tag{2.69}$$

where the reference value changes every 10 simulation steps.

For this experiment, the control-input weighting was selected as

$$R = 0.1 \tag{2.70}$$

Also, in our example, the prediction horizon is chosen to be the same as the control horizon. So,

$$T = I, \quad U_{full} = U_c \tag{2.71}$$

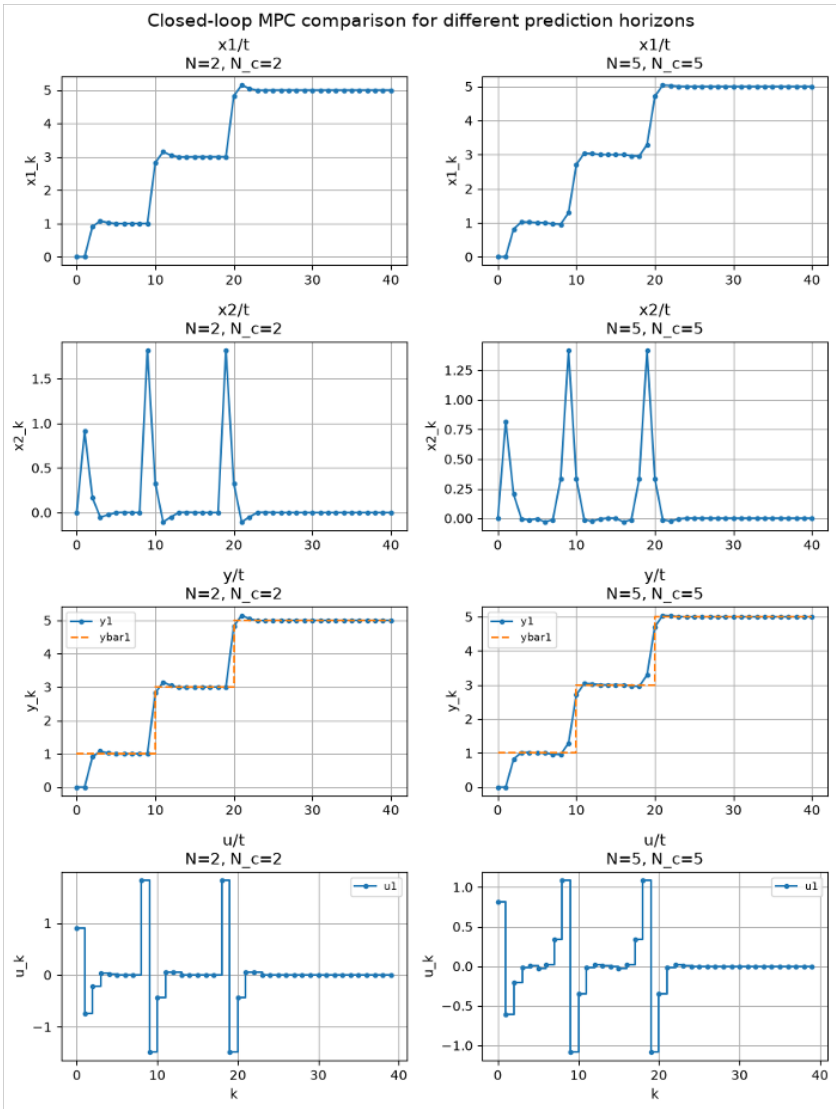


Figure 2.4: Closed-loop tracking performance for $R = 5.0$, Left-side is when N=2 with the averaged RMSE 0.238485, right-side is the case when N=5 with the averaged RMSE 0.242733.

CHAPTER 3

Bilinear Model

3.1 Bilinear System

In the bilinear formulation considered in this work, the system matrices depend on a time-varying variable v_k. The discrete-time state-space model is written as

$$x_{k+1} = A(v_k)x_k + B(v_k)u_k \tag{3.1}$$
$$y_k = Cx_k \tag{3.2}$$

For compactness, the time-varying system matrices are denoted by

$$A_k = A(v_k), \qquad B_k = B(v_k) \tag{3.3}$$

The state equation can therefore be written as

$$x_{k+1} = A_k x_k + B_k u_k \tag{3.4}$$

Unlike the LTI case, where A and B remain constant over the prediction horizon, the matrices A_k and B_k may change at every prediction step according to the corresponding value of v_k.

3.1.1 State and Output Prediction

Starting from the initial state x_0, the first predicted state is

$$x_1 = A_0 x_0 + B_0 u_0 \tag{3.5}$$

The corresponding output is

$$y_1 = Cx_1 = CA_0 x_0 + CB_0 u_0 \tag{3.6}$$

At the second prediction step,

$$x_2 = A_1 x_1 + B_1 u_1 \tag{3.7}$$
$$= A_1(A_0 x_0 + B_0 u_0) + B_1 u_1 \tag{3.8}$$
$$= A_1 A_0 x_0 + A_1 B_0 u_0 + B_1 u_1 \tag{3.9}$$

Therefore,

$$y_2 = Cx_2 \tag{3.10}$$
$$= CA_1A_0x_0 + CA_1B_0u_0 + CB_1u_1 \tag{3.11}$$

Similarly, the third predicted state is

$$x_3 = A_2x_2 + B_2u_2 \tag{3.12}$$
$$= A_2A_1A_0x_0 + A_2A_1B_0u_0 + A_2B_1u_1 + B_2u_2 \tag{3.13}$$

and hence

$$y_3 = CA_2A_1A_0x_0 + CA_2A_1B_0u_0 + CA_2B_1u_1 + CB_2u_2 \tag{3.14}$$

Following the same recursive structure, the state at prediction step k can be expressed as

$$x_k = \left(\prod_{j=0}^{k-1} A_j\right) x_0 + \sum_{i=0}^{k-1} \left(\prod_{j=i+1}^{k-1} A_j\right) B_i u_i \tag{3.15}$$

where the matrix products are ordered such that

$$\prod_{j=0}^{k-1} A_j = A_{k-1}A_{k-2}\cdots A_0 \tag{3.16}$$

For $i = k-1$, the product multiplying $B_{k-1}u_{k-1}$ is interpreted as the identity matrix.

Using $y_k = Cx_k$, the corresponding predicted output becomes

$$y_k = C\left(\prod_{j=0}^{k-1} A_j\right) x_0 + \sum_{i=0}^{k-1} C\left(\prod_{j=i+1}^{k-1} A_j\right) B_i u_i \tag{3.17}$$

3.1.2 Stacked Prediction Formulation

As in the LTI case, the future outputs and control inputs over a prediction horizon N are collected into stacked vectors:

$$Y = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_N \end{bmatrix}, \qquad U = \begin{bmatrix} u_0 \\ u_1 \\ \vdots \\ u_{N-1} \end{bmatrix} \tag{3.18}$$

Using the previously derived output equations, the stacked prediction can be written as

$$Y = \Phi(V)x_0 + \Gamma(V)U, \tag{3.19}$$

where

$$V = \begin{bmatrix} v_0 & v_1 & \cdots & v_{N-1} \end{bmatrix}^T \tag{3.20}$$

contains the values determining the system matrices over the prediction horizon. The state-dependent prediction matrix is

$$\Phi(V) = \begin{bmatrix} CA_0 \\ CA_1A_0 \\ CA_2A_1A_0 \\ \vdots \\ CA_{N-1}A_{N-2}\cdots A_0 \end{bmatrix} \tag{3.21}$$

The input prediction matrix is

$$\Gamma(V) = \begin{bmatrix} CB_0 & 0 & 0 & \cdots & 0 \\ CA_1B_0 & CB_1 & 0 & \cdots & 0 \\ CA_2A_1B_0 & CA_2B_1 & CB_2 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ CA_{N-1}\cdots A_1B_0 & CA_{N-1}\cdots A_2B_1 & CA_{N-1}\cdots A_3B_2 & \cdots & CB_{N-1} \end{bmatrix} \tag{3.22}$$

Consequently, the compact prediction equation is

$$\boxed{Y = \Phi(V)x_0 + \Gamma(V)U} \tag{3.23}$$

3.2 Optimal Control Problem

For the bilinear system, both the control sequence U and the scheduling-variable sequence V are treated as optimization variables. The decision vector is therefore defined as

$$z = \begin{bmatrix} U \\ V \end{bmatrix} \tag{3.24}$$

Using the bilinear stacked prediction model,

$$Y_{\text{pred}} = \Phi(V)x_0 + \Gamma(V)U \tag{3.25}$$

the tracking error is defined as

$$e = Y_{\text{pred}} - \bar{Y} \tag{3.26}$$

The quadratic objective function is then written as

$$\phi(z) = e^T Q e + U^T R_U U + V^T R_V V, \tag{3.27}$$

where $Q \succeq 0$ penalizes the output tracking error, while $R_U \succ 0$ and $R_V \succ 0$ penalize the magnitudes of the control and scheduling-variable sequences, respectively.

Substituting the prediction model into the cost function gives

$$\begin{aligned}\phi(U,V) = &\left(\Phi(V)x_0 + \Gamma(V)U - \bar{Y}\right)^T Q \left(\Phi(V)x_0 + \Gamma(V)U - \bar{Y}\right) \\ &+ U^T R_U U + V^T R_V V\end{aligned} \tag{3.28}$$

The optimal control and scheduling sequences are therefore obtained from

$$\boxed{(U^*, V^*) = \arg\min_{U,V} \phi(U,V)} \tag{3.29}$$

In the LTI case, the prediction matrices Φ and Γ are constant. As a result, the cost function is quadratic in U, and the optimal solution can be obtained analytically by setting the gradient equal to zero.

For the bilinear system, however, both $\Phi(V)$ and $\Gamma(V)$ depend on the optimization variable V. Consequently, the predicted output depends nonlinearly on the joint decision vector $z = [U^T \ V^T]^T$. The resulting optimization problem is therefore generally nonlinear and does not admit the same closed-form solution used for the LTI system.

3.2.1 Numerical Solution Using SLSQP

The nonlinear optimization problem in (3.29) is solved numerically using Sequential Least Squares Programming (SLSQP).

SLSQP is a gradient-based constrained optimization method that solves a sequence of local quadratic approximations of the original nonlinear problem. At each iteration, the algorithm updates the decision variables while attempting to reduce the objective function and satisfy the imposed constraints.

The optimization variable is

$$z = \begin{bmatrix} U \\ V \end{bmatrix} \tag{3.30}$$

with an initial guess

$$z^{(0)} = \begin{bmatrix} U^{(0)} \\ V^{(0)} \end{bmatrix} \tag{3.31}$$

At each SLSQP iteration, the current sequence V is used to construct the time-varying matrices

$$A_k = A(v_k), \qquad B_k = B(v_k) \tag{3.32}$$

and consequently the prediction matrices

$$\Phi(V), \qquad \Gamma(V) \tag{3.33}$$

The predicted output is then evaluated as

$$Y_{\text{pred}} = \Phi(V)x_0 + \Gamma(V)U \tag{3.34}$$

and the objective function $\phi(U, V)$ is computed.

The optimizer repeatedly updates U and V until a convergence criterion is satisfied, resulting in the numerical solution

$$z^* = \begin{bmatrix} U^* \\ V^* \end{bmatrix} \tag{3.35}$$

3.3 Open-Loop Implementation

The open-loop implementation of the bilinear controller follows the same general structure as the open-loop implementation introduced for the LTI system. However, the optimal control problem is solved numerically for both U and V, rather than analytically for U alone.

The implementation procedure can therefore be summarized as follows:

1. Select the prediction horizon N, reference trajectory $\bar{Y}$, weighting matrices Q, R_U, R_V, and the initial state x_0.
2. Define the bounds and initial guesses for the optimization variables U and V.
3. For a candidate sequence V, construct the time-varying matrices

$$A_k = A(v_k), \qquad B_k = B(v_k)$$

4. Construct the prediction matrices $\Phi(V)$ and $\Gamma(V)$.
5. Construct the nonlinear objective function $\phi(U, V)$.
6. Solve the optimization problem numerically using SLSQP to obtain U^* and V^*.
7. Apply the components of U^* and V^* sequentially to the bilinear system.
8. Compare the resulting output trajectory with the reference trajectory.

At each simulation step, the optimized values are applied according to

$$x_{k+1} = A(v_k^*)x_k + B(v_k^*)u_k^* \tag{3.36}$$
$$y_k = Cx_k \tag{3.37}$$

Since the controller operates in open loop, the optimization is not updated after the simulation begins. Therefore, the sequences U^* and V^* are determined entirely from the initial state and the prescribed reference trajectory.

3.3.1 Results

The bilinear open-loop controller was evaluated using the system

$$x_{k+1} = A(v_k)x_k + B(v_k)u_k, \qquad y_k = Cx_k \tag{3.38}$$

with

$$A(v) = \begin{bmatrix} 0.9 & v \\ 0.0 & 0.5 \end{bmatrix} \tag{3.39}$$

$$B(v) = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \qquad C = \begin{bmatrix} 1.0 & 0.0 \end{bmatrix} \tag{3.40}$$

The initial state was selected as

$$x_0 = \begin{bmatrix} 1.0 \\ 0.0 \end{bmatrix} \tag{3.41}$$

The optimization bounds were chosen as

$$-5.0 \leq u_k \leq 5.0 \tag{3.42}$$
$$-10.0 \leq v_k \leq 10.0 \tag{3.43}$$

The control penalties were selected as

$$R_U = 0.1, \qquad R_V = 0.1 \tag{3.44}$$

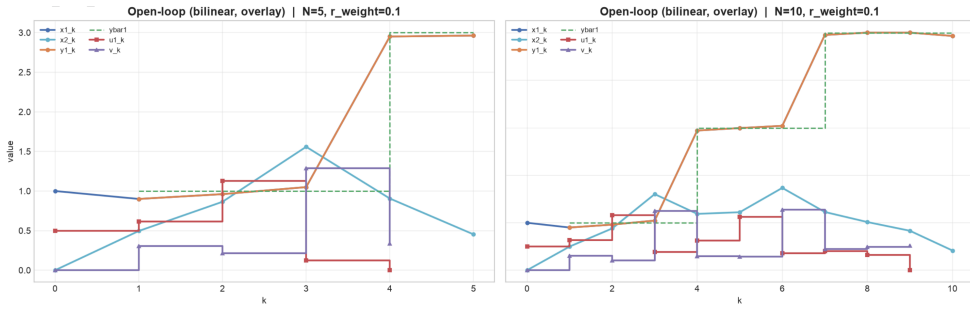


Figure 3.1: Open-loop output tracking performance of the bilinear system, left-side is the case when N=5, and right-side is the case when N=10.

3.4 Closed-Loop Implementation

The closed-loop implementation of the bilinear controller follows the same receding-horizon principle introduced for the LTI system. However, in the bilinear case, both the control sequence U and the scheduling-variable sequence V are optimized at every sampling instant.

At time step k, the current state x_k is used together with the reference trajectory over the prediction horizon. The nonlinear optimization problem is then solved numerically using SLSQP. Only the first elements of the optimal sequences are applied to the system. At the next time step, the state is updated and the complete optimization problem is solved again.

Only the first control values are applied to the system. Using the selection matrix Π,

$$u_k = \Pi U_c^* \tag{3.45}$$

$$v_k = \Pi V_c^* \tag{3.46}$$

After the state is updated, the prediction horizon is shifted forward by one step and the optimization problem is solved again using the new state x_{k+1}.

The closed-loop bilinear control procedure can therefore be summarized as follows:

1. Select N, N_c, N_{end}, Q, R_U, R_V, the reference trajectory, and x_0.
2. Define the bounds and initial guesses for the optimization variables U_c and V_c, and construct the control-horizon mapping matrix T.
3. At time step k, construct the reference vector $\bar{Y}_k$ over the current prediction horizon.

4. For the current candidate sequence V_c, construct $V_{\text{full}} = TV_c$ and the corresponding time-varying system matrices $A_i = A(v_i), \quad B_i = B(v_i)$.
5. Construct the prediction matrices $\Phi(V_{\text{full}})$ and $\Gamma(V_{\text{full}})$.
6. Solve the nonlinear optimization problem using SLSQP to obtain U_c^* and V_c^*.
7. Apply only the first elements of the optimal sequences: $u_k = \Pi U_c^*, \quad v_k = \Pi V_c^*$.
8. Update the system state and output: $x_{k+1} = A(v_k)x_k + B(v_k)u_k, \quad y_k = Cx_k$.
9. Shift the prediction horizon and repeat the optimization for $k+1$ until $k = N_{\text{end}} - 1$.

3.4.1 Results

The closed-loop bilinear controller was evaluated using the same system and parameter settings as in the open-loop implementation.

However, the sequences U^* and V^* are not computed only once. Instead, a new nonlinear optimization problem is solved at every sampling instant using the current state of the system.

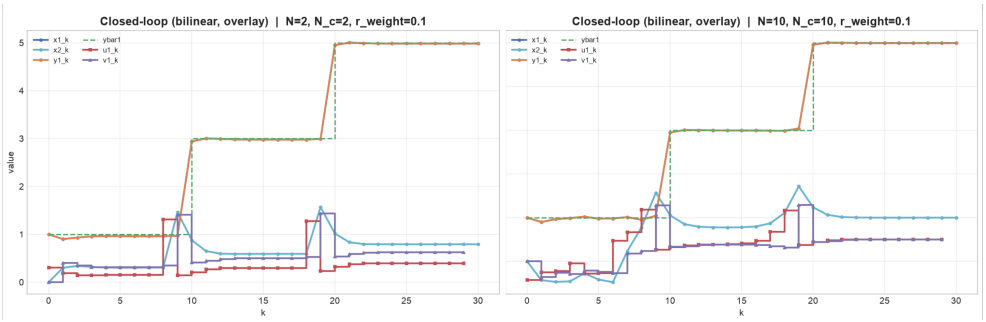


Figure 3.2: Closed-loop output tracking performance of the bilinear system, left-side is the case when N=2, and right-side is the case when N=10.

CHAPTER 4

Stochastic Model

4.1 Stochastic System

A stochastic discrete-time linear system can be written as

$$x_{k+1} = Ax_k + Bu_k + w_k \tag{4.1}$$
$$y_k = Cx_k + v_k \tag{4.2}$$

where w_k represents the process noise and v_k represents the measurement noise. The noise terms are assumed to be zero-mean Gaussian random variables,

$$w_k \sim \mathcal{N}(0, R_w) \tag{4.3}$$
$$v_k \sim \mathcal{N}(0, R_v) \tag{4.4}$$

and the initial state is assumed to be Gaussian distributed as

$$x_0 \sim \mathcal{N}(\mu_0, P_0) \tag{4.5}$$

Under these assumptions, the state distribution at time step k is characterized by its mean and covariance:

$$x_k \sim \mathcal{N}(\mu_k, P_k) \tag{4.6}$$

4.1.1 State Mean

The mean of the state is defined as

$$\mu_k = E[x_k] \tag{4.7}$$

Taking the expectation of the state equation gives

$$\mu_{k+1} = E[x_{k+1}] \tag{4.8}$$
$$= E[Ax_k + Bu_k + w_k] \tag{4.9}$$

Using the linearity of expectation,

$$\mu_{k+1} = AE[x_k] + BE[u_k] + E[w_k] \tag{4.10}$$

Assuming that the control input u_k is deterministic and using

$$E[w_k] = 0 \tag{4.11}$$

the state mean evolves according to

$$\boxed{\mu_{k+1} = A\mu_k + Bu_k} \tag{4.12}$$

Therefore, the control input shifts the mean of the state distribution.

4.1.2 State Covariance

The state covariance is defined as

$$P_k = E\left[(x_k - \mu_k)(x_k - \mu_k)^T\right] \tag{4.13}$$

From (4.1) and (4.12),

$$\begin{aligned} x_{k+1} - \mu_{k+1} &= Ax_k + Bu_k + w_k - (A\mu_k + Bu_k) && (4.14)\\ &= A(x_k - \mu_k) + w_k && (4.15) \end{aligned}$$

Hence,

$$\begin{aligned} P_{k+1} &= E\left[(x_{k+1} - \mu_{k+1})(x_{k+1} - \mu_{k+1})^T\right] && (4.16)\\ &= E\left[(A(x_k - \mu_k) + w_k)\left(A(x_k - \mu_k) + w_k\right)^T\right] && (4.17) \end{aligned}$$

Expanding the product gives

$$\begin{aligned} P_{k+1} = {} & AE\left[(x_k - \mu_k)(x_k - \mu_k)^T\right] A^T \\ & + E[w_k w_k^T] \\ & + AE\left[(x_k - \mu_k)w_k^T\right] \\ & + E\left[w_k(x_k - \mu_k)^T\right] A^T \end{aligned} \tag{4.18}$$

Assuming that the process noise w_k is independent of the state deviation $x_k - \mu_k$, the cross terms are zero. Since

$$E[w_k w_k^T] = R_w \tag{4.19}$$

the covariance recursion becomes

$$\boxed{P_{k+1} = AP_kA^T + R_w} \tag{4.20}$$

Thus, the state uncertainty propagates through the system dynamics and increases due to the process noise.

4.1.3 Output Mean

The stochastic output equation is

$$y_k = Cx_k + v_k \tag{4.21}$$

The mean of the output is therefore

$$\begin{aligned} \mu_{y,k} &= E[y_k] && (4.22)\\ &= E[Cx_k + v_k] && (4.23) \end{aligned}$$

Using the linearity of expectation and

$$E[v_k] = 0 \tag{4.24}$$

we obtain

$$\boxed{\mu_{y,k} = C\mu_k} \tag{4.25}$$

4.1.4 Output Covariance

The output covariance is defined as

$$P_{y,k} = E\left[(y_k - \mu_{y,k})(y_k - \mu_{y,k})^T\right] \tag{4.26}$$

Using

$$x_k = \mu_k + e_k \tag{4.27}$$

where

$$e_k = x_k - \mu_k \qquad E[e_k] = 0 \tag{4.28}$$

the output can be expressed as

$$\begin{aligned} y_k &= C(\mu_k + e_k) + v_k && (4.29)\\ &= C\mu_k + Ce_k + v_k && (4.30) \end{aligned}$$

Since

$$\mu_{y,k} = C\mu_k \tag{4.31}$$

the output deviation from its mean is

$$y_k - \mu_{y,k} = Ce_k + v_k \tag{4.32}$$

Therefore,

$$P_{y,k} = E\left[(Ce_k + v_k)(Ce_k + v_k)^T\right] \tag{4.33}$$

Expanding the expression gives

$$\begin{aligned} P_{y,k} = {} & CE[e_k e_k^T]C^T + E[v_k v_k^T] \\ & + CE[e_k v_k^T] + E[v_k e_k^T]C^T \end{aligned} \tag{4.34}$$

Assuming that the measurement noise v_k is independent of the state error e_k, the cross terms vanish. Since

$$E[e_k e_k^T] = P_k \tag{4.35}$$

and

$$E[v_k v_k^T] = R_v, \tag{4.36}$$

the output covariance becomes

$$\boxed{P_{y,k} = CP_kC^T + R_v} \tag{4.37}$$

Therefore, the stochastic state and output distributions are completely characterized by the following recursive equations:

$$\mu_{k+1} = A\mu_k + Bu_k, \tag{4.38}$$
$$P_{k+1} = AP_kA^T + R_w, \tag{4.39}$$
$$\mu_{y,k} = C\mu_k, \tag{4.40}$$
$$P_{y,k} = CP_kC^T + R_v \tag{4.41}$$

Hence,

$$x_k \sim \mathcal{N}(\mu_k, P_k) \tag{4.42}$$

while the corresponding output distribution is

$$y_k \sim \mathcal{N}\left(\mu_{y,k}, P_{y,k}\right) \tag{4.43}$$

4.2 Model with Chance Constraints

In stochastic systems, the output cannot in general be guaranteed to remain inside deterministic bounds at every time step because of the presence of uncertainty. Instead, the constraints can be imposed probabilistically using chance constraints.

For each prediction step k, the output is required to satisfy

$$P\left(y_{\min} \leq y_k \leq y_{\max}\right) \geq 1 - \alpha, \qquad k = 1, \ldots, N \tag{4.44}$$

where α denotes the allowed probability of constraint violation.

The total violation probability is divided between the upper and lower bounds according to

$$\alpha_{\text{upper}} + \alpha_{\text{lower}} = \alpha \tag{4.45}$$

The two-sided chance constraint can therefore be handled through separate upper and lower probabilistic constraints.

Upper Chance Constraint

The upper output constraint is

$$P\left(y_k \leq y_{\max}\right) \geq 1 - \alpha_{\text{upper}} \tag{4.46}$$

Since

$$y_k \sim \mathcal{N}\left(\mu_{y,k}, \sigma_{y,k}^2\right) \tag{4.47}$$

the output can be standardized as

$$Z = \frac{y_k - \mu_{y,k}}{\sigma_{y,k}}, \qquad Z \sim \mathcal{N}(0,1) \tag{4.48}$$

Therefore,

$$P\left(y_k \leq y_{\max}\right) = P\left(Z \leq \frac{y_{\max} - \mu_{y,k}}{\sigma_{y,k}}\right) \tag{4.49}$$

Using the cumulative distribution function $\Phi(\cdot)$ of the standard normal distribution,

$$\Phi\left(\frac{y_{\max} - \mu_{y,k}}{\sigma_{y,k}}\right) \geq 1 - \alpha_{\text{upper}} \tag{4.50}$$

Applying the inverse standard normal cumulative distribution function yields

$$\frac{y_{\max} - \mu_{y,k}}{\sigma_{y,k}} \geq \Phi^{-1}\left(1 - \alpha_{\text{upper}}\right) \tag{4.51}$$

Rearranging gives

$$\mu_{y,k} \leq y_{\max} - \Phi^{-1}\left(1 - \alpha_{\text{upper}}\right) \sigma_{y,k} \tag{4.52}$$

Lower Chance Constraint

Similarly, the lower output constraint is

$$P\left(y_k \geq y_{\min}\right) \geq 1 - \alpha_{\text{lower}} \tag{4.53}$$

Following the same standardization procedure gives

$$\mu_{y,k} \geq y_{\min} + \Phi^{-1}\left(1 - \alpha_{\text{lower}}\right) \sigma_{y,k} \tag{4.54}$$

Reformulation

From the stochastic output model,

$$\mu_{y,k} = C\mu_k \tag{4.55}$$

and

$$P_{y,k} = CP_kC^T + R_v \tag{4.56}$$

For a scalar output, the standard deviation is therefore

$$\sigma_{y,k} = \sqrt{CP_kC^T + R_v} \tag{4.57}$$

Substituting these expressions into the deterministic chance constraints gives

$$C\mu_k \leq y_{\max} - \Phi^{-1}\left(1 - \alpha_{\text{upper}}\right) \sqrt{CP_kC^T + R_v} \tag{4.58}$$

and

$$C\mu_k \geq y_{\min} + \Phi^{-1}\left(1 - \alpha_{\text{lower}}\right) \sqrt{CP_kC^T + R_v} \tag{4.59}$$

The resulting deterministic limits are referred to as the *tight bounds*. They are defined as

$$\boxed{\tilde{y}_{\min,k} = y_{\min} + \Phi^{-1}\left(1 - \alpha_{\text{lower}}\right) \sigma_{y,k}} \tag{4.60}$$

and

$$\boxed{\tilde{y}_{\max,k} = y_{\max} - \Phi^{-1}\left(1 - \alpha_{\text{upper}}\right) \sigma_{y,k}} \tag{4.61}$$

Hence, the original probabilistic constraint can be enforced through the deterministic condition

$$\boxed{\tilde{y}_{\min,k} \leq C\mu_k \leq \tilde{y}_{\max,k}} \tag{4.62}$$

The tight bounds shrink the original feasible output interval in order to account for stochastic uncertainty. The amount of tightening depends on the output standard deviation $\sigma_{y,k}$ and on the selected violation probabilities.

A larger uncertainty results in a larger distance between the original bounds and the tight bounds. Similarly, selecting a smaller violation probability increases that difference.

4.3 Stacked Forms Derivation

For optimization over a finite prediction horizon, the state means, output means, and control inputs are collected into stacked vectors. This allows the stochastic control problem to be expressed in a compact matrix form.

4.3.1 Stacked Prediction Matrices

The mean-state dynamics are given by

$$\mu_{k+1} = A\mu_k + Bu_k \tag{4.63}$$

Starting from the initial mean μ_0, the first predicted mean is

$$\mu_1 = A\mu_0 + Bu_0 \tag{4.64}$$

The second predicted mean is

$$\begin{aligned} \mu_2 &= A\mu_1 + Bu_1 && (4.65)\\ &= A(A\mu_0 + Bu_0) + Bu_1 && (4.66)\\ &= A^2\mu_0 + ABu_0 + Bu_1 && (4.67) \end{aligned}$$

Similarly,

$$\mu_3 = A^3\mu_0 + A^2Bu_0 + ABu_1 + Bu_2 \tag{4.68}$$

The state means over a prediction horizon N are stacked as

$$M = \begin{bmatrix} \mu_1 \\ \mu_2 \\ \vdots \\ \mu_N \end{bmatrix} \tag{4.69}$$

while the control inputs are collected in

$$U = \begin{bmatrix} u_0 \\ u_1 \\ \vdots \\ u_{N-1} \end{bmatrix} \tag{4.70}$$

The stacked mean-state prediction can therefore be written as

$$\boxed{M = \mathcal{A}\mu_0 + \mathcal{B}U} \tag{4.71}$$

where

$$\mathcal{A} = \begin{bmatrix} A \\ A^2 \\ \vdots \\ A^N \end{bmatrix} \tag{4.72}$$

and

$$\mathcal{B} = \begin{bmatrix} B & 0 & \cdots & 0 \\ AB & B & \cdots & 0 \\ A^2B & AB & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ A^{N-1}B & A^{N-2}B & \cdots & B \end{bmatrix} \tag{4.73}$$

The output mean at each prediction step is

$$\mu_{y,k} = C\mu_k \tag{4.74}$$

The stacked output-mean vector is therefore defined as

$$M_y = \begin{bmatrix} \mu_{y,1} \\ \mu_{y,2} \\ \vdots \\ \mu_{y,N} \end{bmatrix} \tag{4.75}$$

Defining the block-diagonal output matrix

$$\mathcal{C} = I_N \otimes C \tag{4.76}$$

the stacked output means satisfy

$$M_y = \mathcal{C}M \tag{4.77}$$

Substituting (4.71) gives

$$M_y = \mathcal{C}\mathcal{A}\mu_0 + \mathcal{C}\mathcal{B}U \tag{4.78}$$

Introducing

$$F = \mathcal{C}\mathcal{A}, \qquad G = \mathcal{C}\mathcal{B} \tag{4.79}$$

the stacked output-mean prediction becomes

$$\boxed{M_y = F\mu_0 + GU} \tag{4.80}$$

4.3.2 Stacked Cost Function

For a stochastic output,

$$y_k = \mu_{y,k} + e_{y,k} \tag{4.81}$$

where

$$E[e_{y,k}] = 0 \tag{4.82}$$

The expected quadratic tracking cost at one prediction step is

$$E\left[(y_k - \bar{y}_k)^T Q (y_k - \bar{y}_k)\right] \tag{4.83}$$

Substituting

$$y_k = \mu_{y,k} + e_{y,k} \tag{4.84}$$

gives

$$\begin{aligned} &E\left[(y_k - \bar{y}_k)^T Q (y_k - \bar{y}_k)\right] \\ &= (\mu_{y,k} - \bar{y}_k)^T Q (\mu_{y,k} - \bar{y}_k) + E\left[e_{y,k}^T Q e_{y,k}\right] \end{aligned} \tag{4.85}$$

Using the trace identity

$$x^T Q x = \operatorname{tr}\left(Q x x^T\right) \tag{4.86}$$

the second term can be written as

$$E\left[e_{y,k}^T Q e_{y,k}\right] = \operatorname{tr}\left(Q E[e_{y,k} e_{y,k}^T]\right) \tag{4.87}$$

$$= \operatorname{tr}\left(Q P_{y,k}\right) \tag{4.88}$$

Since

$$P_{y,k} = C P_k C^T + R_v \tag{4.89}$$

the expected tracking cost becomes

$$
\begin{aligned}
&E\left[(y_k - \bar{y}_k)^T Q (y_k - \bar{y}_k)\right] \\
&= (\mu_{y,k} - \bar{y}_k)^T Q (\mu_{y,k} - \bar{y}_k) + \operatorname{tr}\left[Q(CP_kC^T + R_v)\right]
\end{aligned} \tag{4.90}
$$

For the additive-noise stochastic system considered here, the covariance recursion

$$
P_{k+1} = AP_kA^T + R_w \tag{4.91}
$$

does not depend on the deterministic control input u_k. Therefore,

$$
\operatorname{tr}\left[Q(CP_kC^T + R_v)\right] \tag{4.92}
$$

is constant with respect to the optimization variable U and can be omitted when determining the optimal control sequence.

The resulting optimization objective is therefore

$$
\phi_{\text{opt}} = \frac{1}{2} \sum_{k=0}^{N-1} \left[(C\mu_{k+1} - \bar{y}_{k+1})^T Q (C\mu_{k+1} - \bar{y}_{k+1}) + u_k^T R u_k\right] \tag{4.93}
$$

Define the stacked reference vector as

$$
\bar{Y}_{\text{ref}} = \begin{bmatrix} \bar{y}_1 \\ \bar{y}_2 \\ \vdots \\ \bar{y}_N \end{bmatrix} \tag{4.94}
$$

and let the stacked weighting matrices be

$$
\bar{Q} = \operatorname{blkdiag}(Q, \ldots, Q) \tag{4.95}
$$

$$
\bar{R} = \operatorname{blkdiag}(R, \ldots, R) \tag{4.96}
$$

Using

$$
M_y = F\mu_0 + GU \tag{4.97}
$$

the stacked cost function becomes

$$
\phi = \frac{1}{2} \left[(F\mu_0 + GU - \bar{Y}_{\text{ref}})^T \bar{Q} (F\mu_0 + GU - \bar{Y}_{\text{ref}}) + U^T \bar{R} U\right] \tag{4.98}
$$

Expanding (4.98) gives

$$
\begin{aligned}
\phi = &\frac{1}{2} U^T \left(G^T \bar{Q} G + \bar{R}\right) U \\
&+ (F\mu_0 - \bar{Y}_{\text{ref}})^T \bar{Q} G\, U + \text{const}
\end{aligned} \tag{4.99}
$$

Define

$$H = G^T \bar{Q} G + \bar{R} \tag{4.100}$$

and

$$f = G^T \bar{Q} \left(F\mu_0 - \bar{Y}_{\text{ref}} \right) \tag{4.101}$$

The cost function can then be written in standard quadratic-programming form as

$$\boxed{\phi(U) = \frac{1}{2} U^T H U + f^T U} \tag{4.102}$$

For the unconstrained problem, the stationary condition is

$$\frac{\partial \phi}{\partial U} = HU + f = 0 \tag{4.103}$$

and therefore

$$U^* = -H^{-1} f \tag{4.104}$$

4.3.3 Stacked Chance Constraints

From the chance-constraint derivation, the deterministic tight bounds at each prediction step are

$$\tilde{y}_{\min,k} \leq \mu_{y,k} \leq \tilde{y}_{\max,k} \tag{4.105}$$

The tight lower and upper bounds are stacked as

$$\tilde{Y}_{\min} = \begin{bmatrix} \tilde{y}_{\min,1} \\ \tilde{y}_{\min,2} \\ \vdots \\ \tilde{y}_{\min,N} \end{bmatrix} \tag{4.106}$$

and

$$\tilde{Y}_{\max} = \begin{bmatrix} \tilde{y}_{\max,1} \\ \tilde{y}_{\max,2} \\ \vdots \\ \tilde{y}_{\max,N} \end{bmatrix} \tag{4.107}$$

Therefore, the stacked deterministic chance constraint becomes

$$\boxed{\tilde{Y}_{\min} \leq M_y \leq \tilde{Y}_{\max}} \tag{4.108}$$

Using

$$M_y = F\mu_0 + GU \tag{4.109}$$

this becomes

$$\tilde{Y}_{\min} \leq F\mu_0 + GU \leq \tilde{Y}_{\max} \tag{4.110}$$

Rearranging with respect to the optimization variable U gives

$$\boxed{\tilde{Y}_{\min} - F\mu_0 \leq GU \leq \tilde{Y}_{\max} - F\mu_0} \tag{4.111}$$

Overview

Combining the stacked cost function and the tight chance constraints, the stochastic optimal control problem can be written as

$$\boxed{\begin{aligned} \min_{U} \quad & \frac{1}{2}U^T HU + f^T U \\ \text{subject to} \quad & \tilde{Y}_{\min} - F\mu_0 \leq GU \leq \tilde{Y}_{\max} - F\mu_0 \end{aligned}} \tag{4.112}$$

4.4 Open-Loop Implementation

The open-loop implementation follows the same general structure as the deterministic LTI open-loop case. The optimal input sequence is computed once at the beginning of the simulation and is not updated afterwards.

For the stochastic system, however, the resulting controller performance is evaluated using Monte Carlo simulation. A single optimal control sequence U^* is first obtained from the stochastic optimization problem. The same sequence is then applied to a large number of independently generated monte carlo trajectories.

The optimization problem is

$$\begin{aligned} \min_{U} \quad & \frac{1}{2}U^T HU + f^T U \\ \text{subject to} \quad & \tilde{Y}_{\min} - F\mu_0 \leq GU \leq \tilde{Y}_{\max} - F\mu_0 \end{aligned} \tag{4.113}$$

Solving this problem yields the optimal open-loop input sequence

$$U^* = \begin{bmatrix} u_0^* \\ u_1^* \\ \vdots \\ u_{N-1}^* \end{bmatrix} \tag{4.114}$$

This sequence is calculated only once using the initial mean and covariance, the reference trajectory, and the tight output bounds.

The stochastic open-loop procedure can therefore be summarized as follows:

1. Select N, μ_0, P_0, Q, R, R_w, R_v, the reference trajectory, and the chance-constraint parameters.
2. Propagate the state covariance and construct the corresponding tight bounds
$$\tilde{Y}_{\min}, \qquad \tilde{Y}_{\max}$$
3. Construct the stacked prediction matrices and the quadratic-programming matrices H and f.
4. Solve the stochastic open-loop optimization problem once to obtain
$$U^*$$
5. Generate n_{MC} independent Monte Carlo realizations of the initial state and stochastic noise.
6. Apply the same optimal sequence U^* to every Monte Carlo trajectory.
7. Simulate all trajectories using the stochastic state and output equations.
8. Evaluate the resulting tracking performance and the empirical constraint-satisfaction probability.

4.4.1 Results

The stochastic open-loop controller was evaluated using a two-state linear system over a prediction horizon of

$$N = 30 \tag{4.115}$$

The system matrices were selected as

$$A = \begin{bmatrix} 0.90 & 1.00 \\ 0.00 & 0.50 \end{bmatrix}, \qquad B = \begin{bmatrix} 0.0 \\ 1.0 \end{bmatrix} \tag{4.116}$$

with both states directly observed at the output:

$$C = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \tag{4.117}$$

The state and input weighting matrices were chosen as

$$Q = I_2, \qquad R = 0.1 \tag{4.118}$$

The process and measurement noise covariance matrices were defined as

$$R_w = \begin{bmatrix} 0.005 & 0 \\ 0 & 0.005 \end{bmatrix} \tag{4.119}$$

and

$$R_v = \begin{bmatrix} 0.005 & 0 \\ 0 & 0.005 \end{bmatrix} \tag{4.120}$$

The initial state distribution was specified by

$$\mu_0 = \begin{bmatrix} 0 \\ 0 \end{bmatrix} \tag{4.121}$$

and

$$P_0 = \begin{bmatrix} 0.01 & 0 \\ 0 & 0.01 \end{bmatrix} \tag{4.122}$$

The output constraints were imposed equally on both outputs as

$$y_{\min} = \begin{bmatrix} -1.7 \\ -1.7 \end{bmatrix} \qquad y_{\max} = \begin{bmatrix} 3.5 \\ 3.5 \end{bmatrix} \tag{4.123}$$

The upper and lower risk allocations were selected as

$$\alpha_{\text{lower}} = 0.01, \qquad \alpha_{\text{upper}} = 0.01 \tag{4.124}$$

These values were used to compute the corresponding tight bounds at each prediction step according to the propagated output covariance.

The reference trajectory was divided into two operating periods. During the first 15 steps, the desired output was

$$\bar{y}^{(1)} = \begin{bmatrix} 3.1 \\ -1.3 \end{bmatrix} \tag{4.125}$$

while during the second 15 steps the reference was changed to

$$\bar{y}^{(2)} = \begin{bmatrix} -1.3 \\ 3.1 \end{bmatrix} \tag{4.126}$$

Thus, the reference schedule is

$$\bar{y}_k = \begin{cases} [\,3.1 \quad -1.3\,]^T, & k = 1, \ldots, 15 \\ [\,-1.3 \quad 3.1\,]^T, & k = 16, \ldots, 30 \end{cases} \tag{4.127}$$

The control input was constrained by

$$-5.0 \leq u_k \leq 5.0 \tag{4.128}$$

A single optimal open-loop input sequence U^* was obtained by solving the stacked stochastic optimization problem. This same sequence was then applied to all Monte Carlo trajectories.

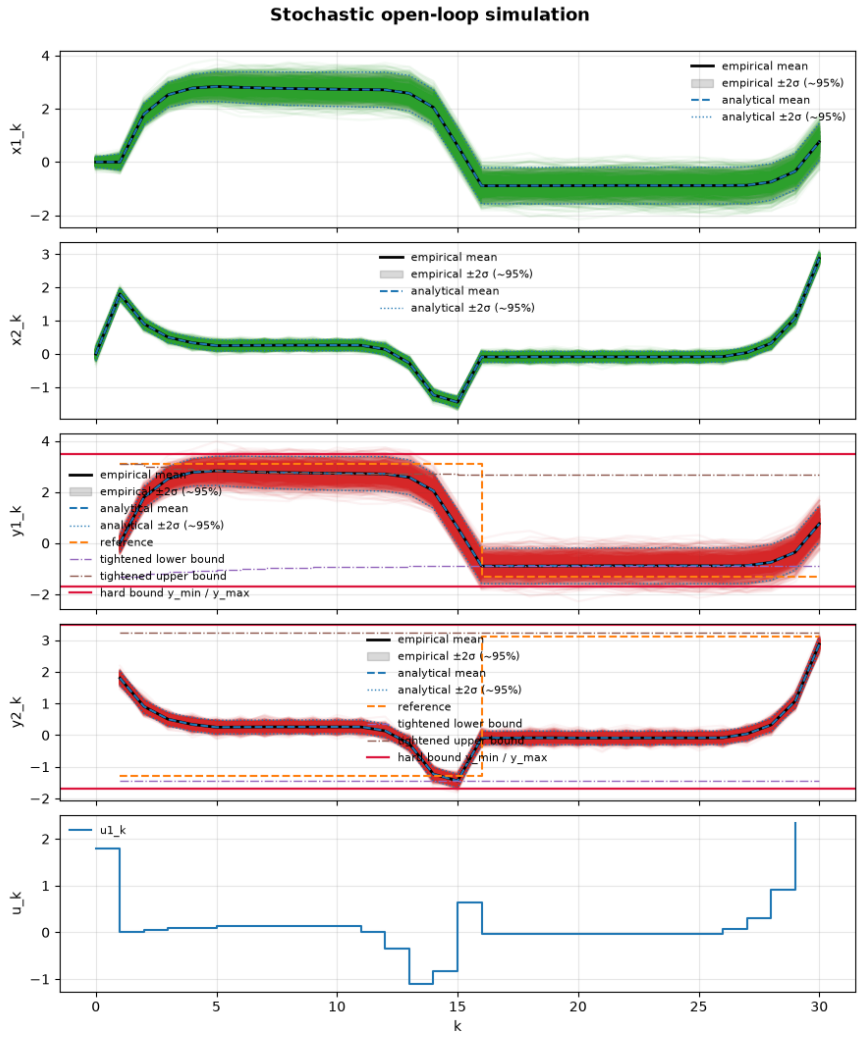


Figure 4.1: Monte Carlo output trajectories for the stochastic open-loop implementation together with the reference trajectory, original output constraints, and tight bounds.

4.5 Closed-Loop Implementation

The closed-loop stochastic implementation follows the same receding-horizon structure used in the LTI closed-loop Case. The main difference is that the controller is evaluated using Monte Carlo simulation, and each stochastic realization solves its own optimization problem using its own current state.

For each Monte Carlo trajectory j, the initial state is sampled independently as

$$x_0^{(j)} \sim \mathcal{N}(\mu_0, P_0) \tag{4.129}$$

Since the realizations start from different initial conditions and are affected by independent process and measurement noise, the state of each trajectory evolves differently.

At every time step k, each trajectory therefore constructs and solves its own model predictive control problem.

The stochastic closed-loop implementation can therefore be summarized as follows:

1. Select N, N_c, N_{end}, Q, R, R_w, R_v, the reference trajectory, and the chance-constraint parameters.
2. Generate n_{MC} independent initial states
$$x_0^{(j)} \sim \mathcal{N}(\mu_0, P_0)$$
3. For each Monte Carlo trajectory j, use its current state $x_k^{(j)}$ as the initial condition of the MPC prediction.
4. Construct the reference vector $\bar{Y}_k$ over the current prediction horizon.
5. Construct the corresponding tight bounds
$$\tilde{Y}_{\min,k}, \qquad \tilde{Y}_{\max,k}$$
6. Solve a separate constrained MPC problem for each Monte Carlo realization to obtain
$$U_k^{*,(j)}$$
7. Apply only the first control action
$$u_k^{(j)} = u_{k|k}^{*,(j)}$$
8. Sample the process and measurement noise and update each stochastic trajectory independently.
9. Shift the prediction horizon and repeat the optimization for every realization at the next time step.
10. Continue until $k = N_{\text{end}} - 1$.

4.5.1 Results

The closed-loop stochastic controller was evaluated using the same system, noise levels, output constraints, reference trajectory, and weighting matrices used in the open-loop experiment.

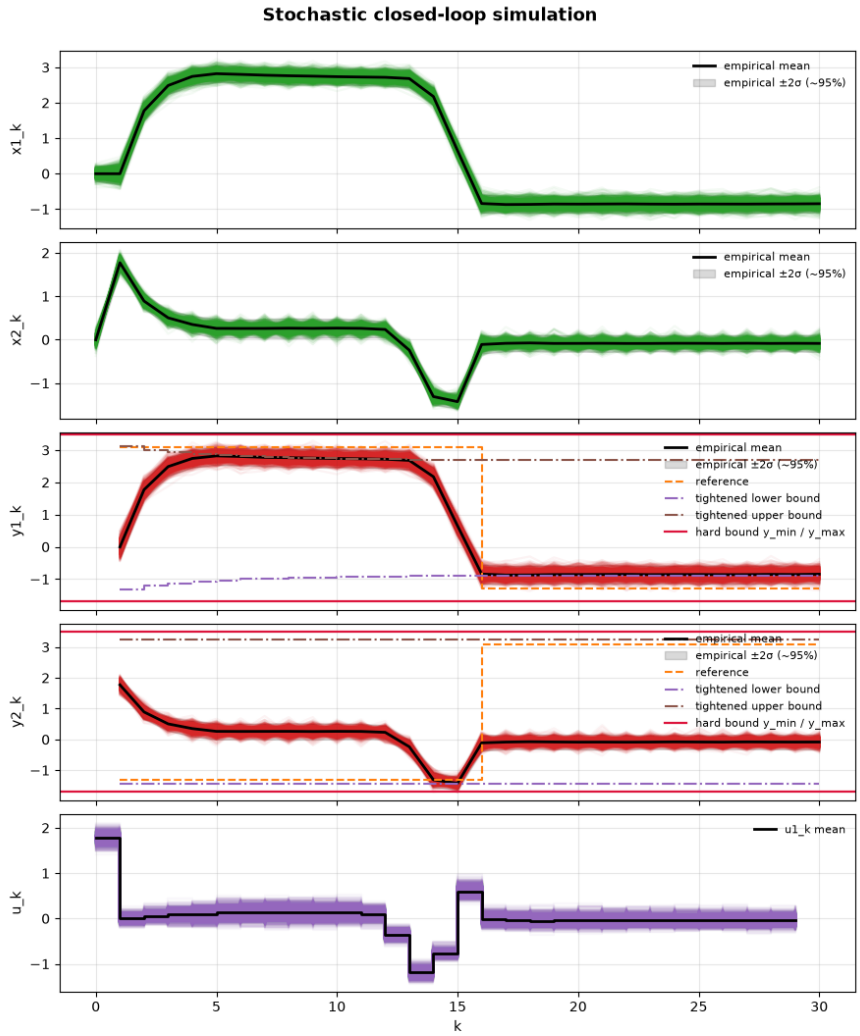


Figure 4.2: Monte Carlo output trajectories for the stochastic closed-loop implementation together with the reference trajectory, original output constraints, and tight bounds.

CHAPTER 5

Stochastic Bilinear Model with Chance Constraints

5.1 Stochastic Bilinear System

A stochastic bilinear system is considered in the form

$$x_{k+1} = A(v_k)x_k + B(v_k)u_k + w_k \tag{5.1}$$
$$y_k = Cx_k + \eta_k \tag{5.2}$$

where u_k denotes the control input and v_k is the bilinear scheduling variable. The stochastic disturbances are assumed to be zero-mean Gaussian,

$$w_k \sim \mathcal{N}(0, R_w) \tag{5.3}$$
$$\eta_k \sim \mathcal{N}(0, R_\eta) \tag{5.4}$$

and the initial state is distributed according to

$$x_0 \sim \mathcal{N}(\mu_0, P_0) \tag{5.5}$$

For compactness, the time-varying system matrices are denoted by

$$A_k = A(v_k), \qquad B_k = B(v_k) \tag{5.6}$$

5.1.1 System Matrices

The state distribution at time step k is characterized by its mean and covariance,

$$x_k \sim \mathcal{N}(\mu_k, P_k) \tag{5.7}$$

The state mean is defined as

$$\mu_k = E[x_k] \tag{5.8}$$

Taking the expectation of (5.1) gives

$$\begin{aligned} \mu_{k+1} &= E\left[A_k x_k + B_k u_k + w_k\right] && (5.9)\\ &= A_k E[x_k] + B_k u_k + E[w_k] && (5.10) \end{aligned}$$

Since the process noise is zero mean,

$$\boxed{\mu_{k+1} = A_k \mu_k + B_k u_k} \tag{5.11}$$

To derive the covariance, the state is decomposed as

$$x_k = \mu_k + e_k, \tag{5.12}$$

where

$$e_k = x_k - \mu_k, \qquad E[e_k] = 0 \tag{5.13}$$

Using the state and mean equations,

$$\begin{aligned} x_{k+1} &= A_k \mu_k + B_k u_k + A_k e_k + w_k && (5.14)\\ \mu_{k+1} &= A_k \mu_k + B_k u_k && (5.15) \end{aligned}$$

and therefore

$$e_{k+1} = A_k e_k + w_k \tag{5.16}$$

The state covariance is

$$P_{k+1} = E\left[e_{k+1} e_{k+1}^T\right] \tag{5.17}$$

Substituting the error dynamics gives

$$\begin{aligned} P_{k+1} &= E\left[(A_k e_k + w_k)(A_k e_k + w_k)^T\right] && (5.18)\\ &= A_k P_k A_k^T + A_k E[e_k w_k^T] + E[w_k e_k^T] A_k^T + R_w && (5.19) \end{aligned}$$

Assuming that the process noise is independent of the state error, the cross terms vanish, giving

$$\boxed{P_{k+1} = A_k P_k A_k^T + R_w} \tag{5.20}$$

Since $A_k = A(v_k)$, the propagated covariance depends on the sequence V. Hence,

$$P_k = P_k(V) \tag{5.21}$$

The output mean follows from

$$y_k = Cx_k + \eta_k \tag{5.22}$$

Taking the expectation gives

$$\boxed{\mu_{y,k} = E[y_k] = C\mu_k} \tag{5.23}$$

The corresponding output covariance is

$$\boxed{P_{y,k} = CP_kC^T + R_\eta} \tag{5.24}$$

Since $P_k = P_k(V)$,

$$P_{y,k} = P_{y,k}(V) \tag{5.25}$$

5.1.2 Stacked Matrix Derivations

The first predicted state mean is

$$\mu_1 = A_0\mu_0 + B_0u_0. \tag{5.26}$$

The second predicted mean becomes

$$\mu_2 = A_1\mu_1 + B_1u_1 \tag{5.27}$$
$$= A_1A_0\mu_0 + A_1B_0u_0 + B_1u_1 \tag{5.28}$$

Similarly,

$$\mu_3 = A_2A_1A_0\mu_0 + A_2A_1B_0u_0 + A_2B_1u_1 + B_2u_2 \tag{5.29}$$

In general,

$$\mu_k = A_{k-1}\cdots A_0\mu_0 + \sum_{j=0}^{k-1} A_{k-1}\cdots A_{j+1}B_ju_j \tag{5.30}$$

where an empty matrix product is interpreted as the identity matrix.
Define the stacked vectors

$$M = \begin{bmatrix} \mu_1 \\ \mu_2 \\ \vdots \\ \mu_N \end{bmatrix}, \qquad U = \begin{bmatrix} u_0 \\ u_1 \\ \vdots \\ u_{N-1} \end{bmatrix}, \qquad V = \begin{bmatrix} v_0 \\ v_1 \\ \vdots \\ v_{N-1} \end{bmatrix} \tag{5.31}$$

The stacked state-mean prediction becomes

$$\boxed{M = \mathcal{A}(V)\mu_0 + \mathcal{B}(V)U} \tag{5.32}$$

The matrix $\mathcal{A}(V)$ is

$$\mathcal{A}(V) = \begin{bmatrix} A_0 \\ A_1 A_0 \\ A_2 A_1 A_0 \\ \vdots \\ A_{N-1} A_{N-2} \cdots A_0 \end{bmatrix} \tag{5.33}$$

while

$$\mathcal{B}(V) = \begin{bmatrix} B_0 & 0 & \cdots & 0 \\ A_1 B_0 & B_1 & \cdots & 0 \\ A_2 A_1 B_0 & A_2 B_1 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ A_{N-1} \cdots A_1 B_0 & A_{N-1} \cdots A_2 B_1 & \cdots & B_{N-1} \end{bmatrix} \tag{5.34}$$

For the outputs, define

$$\mathcal{C} = I_N \otimes C. \tag{5.35}$$

The stacked output-mean vector is

$$M_y = \begin{bmatrix} \mu_{y,1} \\ \mu_{y,2} \\ \vdots \\ \mu_{y,N} \end{bmatrix} = \mathcal{C}M \tag{5.36}$$

Therefore,

$$M_y = \mathcal{C}\mathcal{A}(V)\mu_0 + \mathcal{C}\mathcal{B}(V)U \tag{5.37}$$

Defining

$$\Phi(V) = \mathcal{C}\mathcal{A}(V), \qquad \Gamma(V) = \mathcal{C}\mathcal{B}(V) \tag{5.38}$$

gives

$$\boxed{M_y = \Phi(V)\mu_0 + \Gamma(V)U} \tag{5.39}$$

5.2 Model with Chance Constraints

The probabilistic output constraint is

$$P\left(y_{\min} \leq y_k \leq y_{\max}\right) \geq 1 - \alpha \tag{5.40}$$

The violation probability is divided as

$$\alpha_{\text{lower}} + \alpha_{\text{upper}} = \alpha \tag{5.41}$$

For the stochastic bilinear system,

$$\mu_{y,k} = C\mu_k(U,V) \tag{5.42}$$

while

$$P_{y,k}(V) = CP_k(V)C^T + R_\eta \tag{5.43}$$

For a scalar output, the corresponding standard deviation is

$$\sigma_{y,k}(V) = \sqrt{CP_k(V)C^T + R_\eta} \tag{5.44}$$

The upper chance constraint becomes

$$C\mu_k(U,V) \leq y_{\max} - \Phi_{\mathcal{N}}^{-1}\left(1 - \alpha_{\text{upper}}\right)\sigma_{y,k}(V) \tag{5.45}$$

while the lower constraint becomes

$$C\mu_k(U,V) \geq y_{\min} + \Phi_{\mathcal{N}}^{-1}\left(1 - \alpha_{\text{lower}}\right)\sigma_{y,k}(V) \tag{5.46}$$

where $\Phi_{\mathcal{N}}^{-1}$ denotes the inverse CDF of the standard normal distribution.
The tight bounds are therefore defined as

$$\boxed{\tilde{y}_{\min,k}(V) = y_{\min} + \Phi_{\mathcal{N}}^{-1}(1 - \alpha_{\text{lower}})\sigma_{y,k}(V)} \tag{5.47}$$

and

$$\boxed{\tilde{y}_{\max,k}(V) = y_{\max} - \Phi_{\mathcal{N}}^{-1}(1 - \alpha_{\text{upper}})\sigma_{y,k}(V)} \tag{5.48}$$

Hence,

$$\boxed{\tilde{y}_{\min,k}(V) \leq C\mu_k(U,V) \leq \tilde{y}_{\max,k}(V)} \tag{5.49}$$

Define

$$\tilde{Y}_{\min}(V) = \begin{bmatrix} \tilde{y}_{\min,1}(V) \\ \vdots \\ \tilde{y}_{\min,N}(V) \end{bmatrix} \tag{5.50}$$

and

$$\tilde{Y}_{\max}(V) = \begin{bmatrix} \tilde{y}_{\max,1}(V) \\ \vdots \\ \tilde{y}_{\max,N}(V) \end{bmatrix} \tag{5.51}$$

Using the stacked output-mean prediction,

$$\boxed{\tilde{Y}_{\min}(V) \leq \Phi(V)\mu_0 + \Gamma(V)U \leq \tilde{Y}_{\max}(V)} \tag{5.52}$$

5.3 Cost Function Derivation

The expected stochastic objective is defined as

$$\phi = \frac{1}{2}E\left[(Y - \bar{Y}_{\text{ref}})^T Q(Y - \bar{Y}_{\text{ref}}) + U^T R_U U + V^T R_V V\right] \tag{5.53}$$

Since U and V are deterministic optimization variables,

$$\begin{aligned} \phi = &\frac{1}{2}E\left[(Y - \bar{Y}_{\text{ref}})^T Q(Y - \bar{Y}_{\text{ref}})\right] \\ &+ \frac{1}{2}U^T R_U U + \frac{1}{2}V^T R_V V \end{aligned} \tag{5.54}$$

The stochastic output is decomposed as

$$Y = M_y + E_y \tag{5.55}$$

where

$$E[E_y] = 0 \tag{5.56}$$

Substitution gives

$$E\left[(Y - \bar{Y}_{\text{ref}})^T Q(Y - \bar{Y}_{\text{ref}})\right] \tag{5.57}$$

$$= (M_y - \bar{Y}_{\text{ref}})^T Q(M_y - \bar{Y}_{\text{ref}}) + E[E_y^T Q E_y] \tag{5.58}$$

Using the trace identity,

$$E[E_y^T Q E_y] = \text{tr}\,(Q P_Y) \tag{5.59}$$

Therefore,

$$\phi = \frac{1}{2}\left[(M_y - \bar{Y}_{\text{ref}})^T Q(M_y - \bar{Y}_{\text{ref}}) + \text{tr}(Q P_Y) + U^T R_U U + V^T R_V V\right] \tag{5.60}$$

For the stochastic bilinear model,

$$M_y = \Phi(V)\mu_0 + \Gamma(V)U \tag{5.61}$$

and the output covariance also depends on V. Hence,

$$\boxed{\begin{aligned}\phi(U,V) = \frac{1}{2}\Big[&\left(\Phi(V)\mu_0 + \Gamma(V)U - \bar{Y}_{\text{ref}}\right)^T Q \left(\Phi(V)\mu_0 + \Gamma(V)U - \bar{Y}_{\text{ref}}\right) \\ &+ U^T R_U U + V^T R_V V + \operatorname{tr}\left(Q P_Y(V)\right)\Big]\end{aligned}} \tag{5.62}$$

5.4 Problem Overview

Combining the stochastic bilinear cost function and the chance constraints gives the complete optimization problem

$$\boxed{\begin{aligned} \min_{U,V} \quad & \phi(U,V) \\ \text{subject to} \quad & \tilde{Y}_{\min}(V) \le \Phi(V)\mu_0 + \Gamma(V)U \le \tilde{Y}_{\max}(V), \\ & U_{\min} \le U \le U_{\max}, \\ & V_{\min} \le V \le V_{\max} \end{aligned}} \tag{5.63}$$

The decision vector can equivalently be written as

$$z = \begin{bmatrix} U \\ V \end{bmatrix} \tag{5.64}$$

5.5 Open-Loop Implementation

The stochastic bilinear open-loop implementation is evaluated using Monte Carlo simulation. A single pair of optimal sequences U^* and V^* is first obtained by solving the nonlinear optimization problem using SLSQP.

The optimization is performed only once:

$$(U^*, V^*) = \arg\min_{U,V} \phi(U,V) \tag{5.65}$$

subject to the chance constraints and the bounds on U and V.

After the optimal sequences have been obtained, the same U^* and V^* are applied to every Monte Carlo realization.

For realization j,

$$x_0^{(j)} \sim \mathcal{N}(\mu_0, P_0) \tag{5.66}$$

and independent stochastic disturbances are sampled as

$$w_k^{(j)} \sim \mathcal{N}(0, R_w) \tag{5.67}$$

$$\eta_k^{(j)} \sim \mathcal{N}(0, R_\eta) \tag{5.68}$$

All trajectories use

$$u_k^{(j)} = u_k^*, \qquad v_k^{(j)} = v_k^* \tag{5.69}$$

The stochastic dynamics are then simulated according to

$$x_{k+1}^{(j)} = A(v_k^*)x_k^{(j)} + B(v_k^*)u_k^* + w_k^{(j)} \tag{5.70}$$

and

$$y_k^{(j)} = Cx_k^{(j)} + \eta_k^{(j)} \tag{5.71}$$

Thus, the optimizer determines one common open-loop pair (U^*, V^*), while the Monte Carlo trajectories differ because of their independently sampled initial conditions and stochastic disturbances.

1. Define N, μ_0, P_0, Q, R_U, R_V, R_w, R_η, the reference trajectory, chance-constraint parameters, and bounds on U and V.

2. Initialize candidate sequences U and V.

3. For each candidate V, construct A_k, B_k, $\Phi(V)$, $\Gamma(V)$, and propagate $P_k(V)$.

4. Construct the V-dependent tight bounds.

5. Evaluate the cost function $\phi(U, V)$.

6. Solve the constrained nonlinear problem using SLSQP to obtain U^* and V^*.

7. Generate the Monte Carlo realizations.

8. Apply the same U^* and V^* to every realization.

9. Evaluate the output trajectories and empirical constraint satisfaction.

5.5.1 Results

The stochastic bilinear open-loop controller was evaluated over a horizon

$$N = 30. \tag{5.72}$$

The bilinear state-transition matrix was defined as

$$A(v) = A_{\text{base}} + vA_v \tag{5.73}$$

where

$$A_{\text{base}} = \begin{bmatrix} 0.90 & 0 \\ 0 & 0.50 \end{bmatrix}, \qquad A_v = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix} \tag{5.74}$$

Hence,

$$A(v) = \begin{bmatrix} 0.90 & v \\ 0 & 0.50 \end{bmatrix} \tag{5.75}$$

The input matrix was

$$B(v) = \begin{bmatrix} 0 \\ 1 \end{bmatrix} \tag{5.76}$$

and both states were directly observed,

$$C = I_2 \tag{5.77}$$

The weighting matrices were

$$Q = I_2, \qquad R_U = 0.1, \qquad R_V = 0.1 \tag{5.78}$$

The process and measurement noise covariance matrices were

$$R_w = R_\eta = \begin{bmatrix} 0.005 & 0 \\ 0 & 0.005 \end{bmatrix} \tag{5.79}$$

The initial state distribution was

$$\mu_0 = \begin{bmatrix} 0 \\ 0 \end{bmatrix}, \qquad P_0 = \begin{bmatrix} 0.01 & 0 \\ 0 & 0.01 \end{bmatrix} \tag{5.80}$$

The output bounds were

$$y_{\min} = \begin{bmatrix} -1.7 \\ -1.7 \end{bmatrix}, \qquad y_{\max} = \begin{bmatrix} 3.5 \\ 3.5 \end{bmatrix} \tag{5.81}$$

with

$$\alpha_{\text{lower}} = \alpha_{\text{upper}} = 0.01 \tag{5.82}$$

The reference trajectory was

$$\bar{y}_k = \begin{cases} [\,3.1 \quad -1.3\,]^T, & k = 1, \ldots, 15, \\ [\,-1.3 \;\; 3.1\,]^T, & k = 16, \ldots, 30 \end{cases} \tag{5.83}$$

The optimization variables were bounded by

$$-5 \leq u_k \leq 5, \tag{5.84}$$

$$-5 \leq v_k \leq 5 \tag{5.85}$$

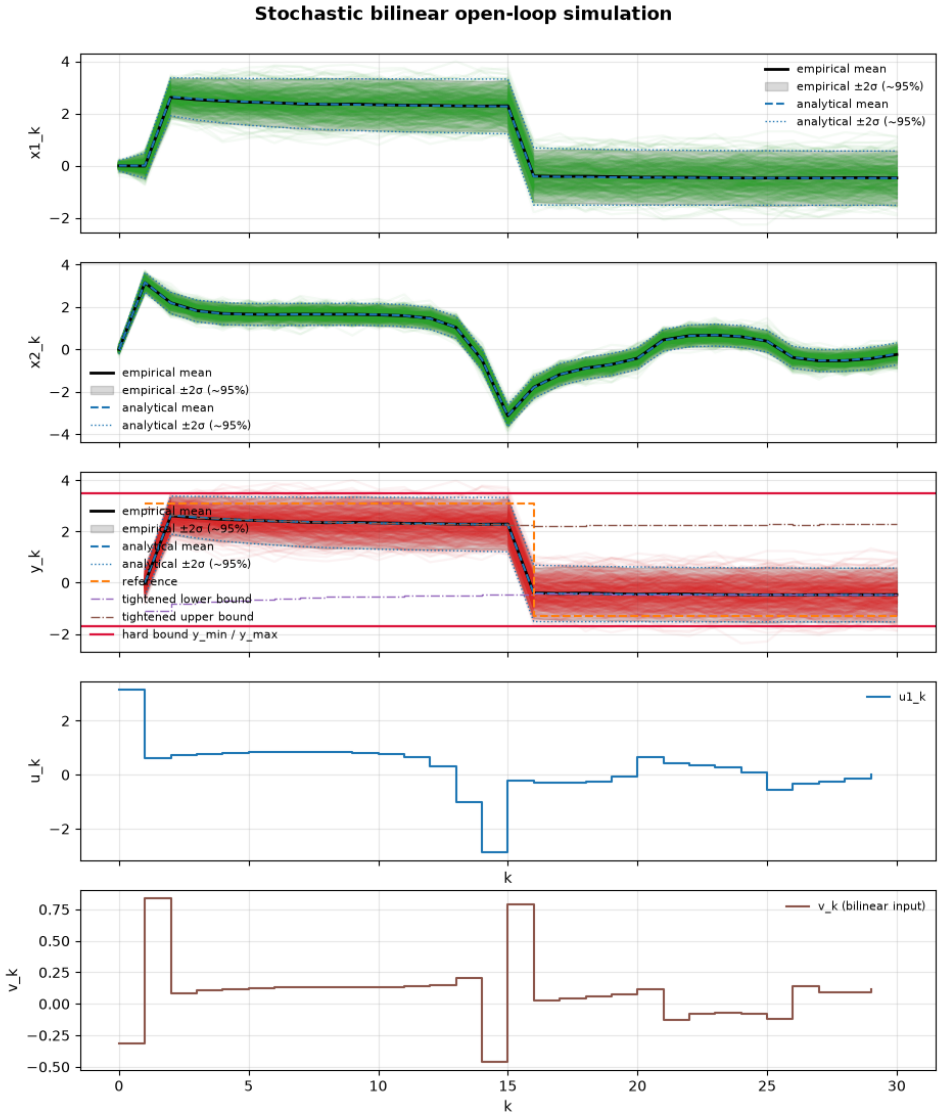


Figure 5.1: Monte Carlo output trajectories for the stochastic bilinear open-loop implementation together with the reference trajectory, original constraints, and tight bounds.

5.6 Closed-Loop Implementation

The stochastic bilinear closed-loop controller is implemented using a receding-horizon Monte Carlo structure similar to the stochastic closed-loop model. But this time for finding optimal U* V* SLSQP solver used.

1. Generate independent initial conditions for all Monte Carlo realizations.

2. At time k, use the current state of each realization as the initial condition for its prediction problem.

3. Construct the current reference trajectory over the prediction horizon.

4. For each candidate V, propagate the bilinear covariance and construct the corresponding tight bounds.

5. Solve the nonlinear constrained optimization problem using SLSQP independently for each realization.

6. Obtain $U_k^{*,(j)}$ and $V_k^{*,(j)}$.

7. Apply only the first pair

$$u_k^{(j)} = u_{k|k}^{*,(j)}, \qquad v_k^{(j)} = v_{k|k}^{*,(j)}$$

8. Sample the stochastic disturbances and update the system.

9. Shift the horizon and repeat the optimization at $k+1$.

5.6.1 Results

The closed-loop experiment used the same stochastic bilinear system with the open-loop case.

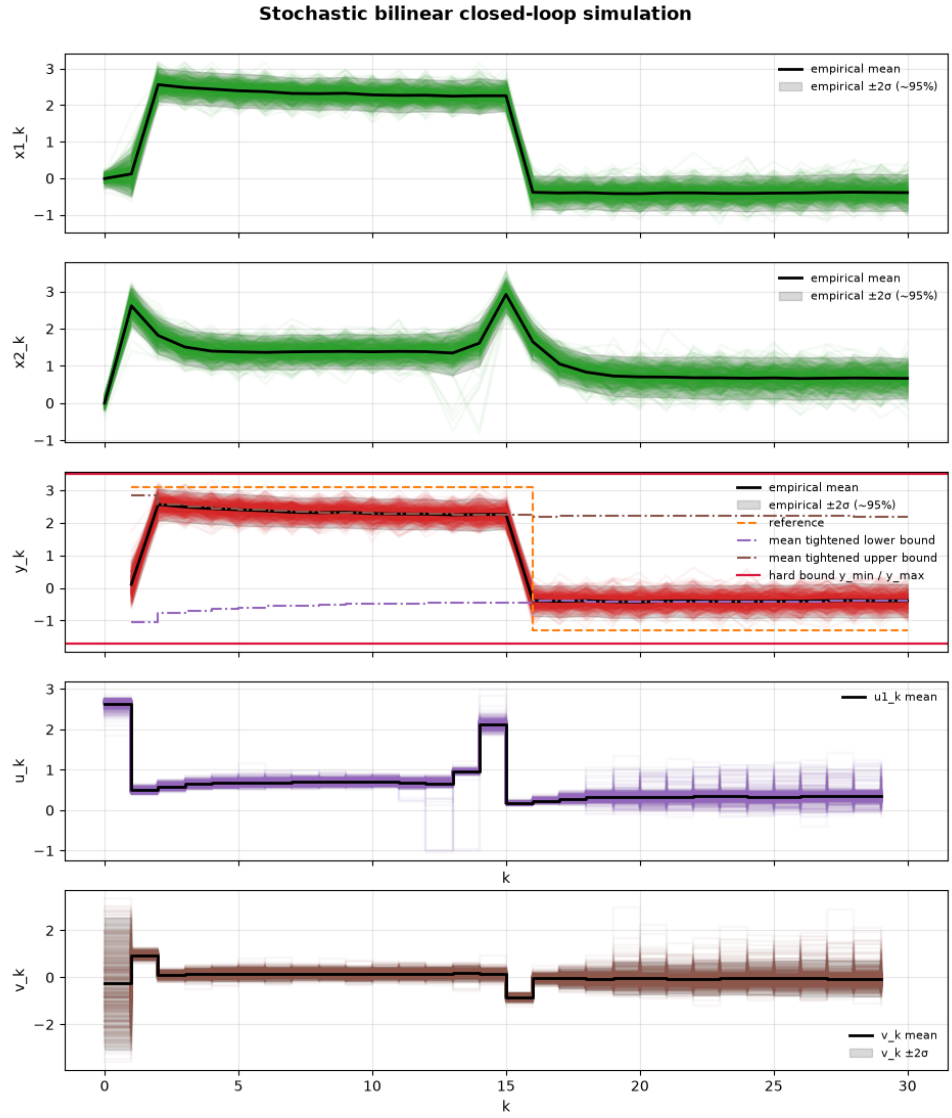


Figure 5.2: Monte Carlo output trajectories for the stochastic bilinear closed-loop implementation.

CHAPTER 6

State Input Analysis

6.1 Analysing Objective

The purpose of the state-input analysis is to investigate how the optimal decision variables change as a function of the system state.

For a two-state system, the initial state is written as

$$x_0 = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \tag{6.1}$$

A rectangular grid is constructed over the (x_1, x_2) state plane. At each grid point, the corresponding optimization problem is solved using that state as the initial condition.

For the linear case, the analysis records the first optimal control input,

$$u_0^* = f(x_1, x_2) \tag{6.2}$$

For the bilinear case, both the first optimal control input and the first optimal bilinear parameter are recorded,

$$u_0^* = f(x_1, x_2) \tag{6.3}$$
$$v_0^* = g(x_1, x_2) \tag{6.4}$$

Only the first optimized values are considered because they represent the immediate control decision associated with a particular state.

The state space is sampled over predefined intervals,

$$x_1 \in [x_{1,\min}, x_{1,\max}], \qquad x_2 \in [x_{2,\min}, x_{2,\max}] \tag{6.5}$$

and the optimization problem is solved independently for every point in this grid. The resulting values form surfaces over the state plane:

$$(x_1, x_2) \longrightarrow u_0^* \tag{6.6}$$

and, for the bilinear system,

$$(x_1, x_2) \longrightarrow v_0^* \tag{6.7}$$

These mappings provide a direct visualization of how the optimal decision changes depending on the location of the system in the state space.

The surfaces are represented using both three-dimensional surface plots and two-dimensional contour plots. The main objective of this analysis is therefore not to study a particular trajectory, but to characterize the state-dependent structure of the optimization problem. In particular, the plots make it possible to identify regions in the state space where the optimal input changes gradually, changes rapidly, approaches its bounds, or exhibits nonlinear behavior.

For the bilinear system, separate plots are produced for u_0^* and v_0^*. This allows the influence of the state on the two optimization variables to be examined independently.

6.2 Results

The state-input relation was evaluated using the stochastic bilinear system

$$A(v) = A_{\text{base}} + vA_v \tag{6.8}$$

with

$$A_{\text{base}} = \begin{bmatrix} 0.90 & 0 \\ 0 & 0.50 \end{bmatrix} \tag{6.9}$$

and

$$A_v = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix} \tag{6.10}$$

Therefore,

$$A(v) = \begin{bmatrix} 0.90 & v \\ 0 & 0.50 \end{bmatrix} \tag{6.11}$$

The input and output matrices were

$$B = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \qquad C = \begin{bmatrix} 1 & 0 \end{bmatrix} \tag{6.12}$$

The prediction horizon was selected as

$$N = 15 \tag{6.13}$$

with weighting matrices

$$Q = 1, \qquad R_U = 0.1, \qquad R_V = 0.1 \tag{6.14}$$

The state-input relation was evaluated over

$$-10 \leq x_1 \leq 10, \qquad -10 \leq x_2 \leq 10 \tag{6.15}$$

using a 50×50 grid.

6.2.1 Optimal Control Input Relation

For every point in the (x_1, x_2) grid, the nonlinear optimization problem was solved and the first optimal control input u_0^* was recorded.

The resulting surface represents

$$u_0^* = f(x_1, x_2) \tag{6.16}$$

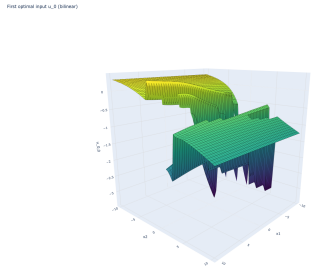


Figure 6.1: First optimal control input u_0^* as a function of the initial state (x_1, x_2).

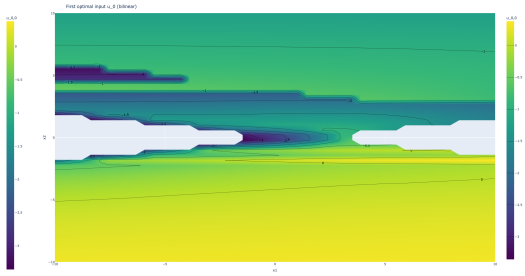


Figure 6.2: Contour representation of u_0^* over the initial-state plane.

The same relation is shown as a contour plot in Figure 6.2. The contour representation provides a two-dimensional view of regions in the state space associated with similar optimal control actions.

6.2.2 Optimal Bilinear Parameter Relation

The same state-space sweep was also used to evaluate the first optimal bilinear parameter. For each initial state, the optimization therefore provides

$$v_0^* = g(x_1, x_2) \tag{6.17}$$

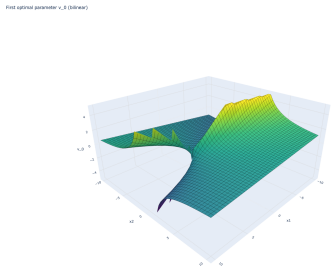


Figure 6.3: First optimal bilinear parameter v_0^* as a function of the initial state (x_1, x_2).

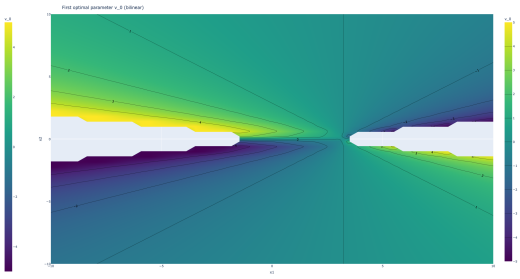


Figure 6.4: Contour representation of v_0^* over the initial-state plane.

The u_0^* surface shows how the immediate control action selected by the optimizer depends on the initial values of x_1 and x_2. Similarly, the v_0^* surface illustrates how the bilinear parameter is adjusted depending on the state.

CHAPTER 7

Kalman Filtering

7.1 Theory Derivation

The Kalman filter is used to estimate the internal state of a stochastic system from noisy measurements. The system considered in this work is

$$x_{t+1} = Ax_t + Bu_t + w_t \tag{7.1}$$
$$y_t = Cx_t + v_t \tag{7.2}$$

where

$$w_t \sim \mathcal{N}(0, R_w), \qquad v_t \sim \mathcal{N}(0, R_v) \tag{7.3}$$

Let

$$Y_t = \{y_0, \ldots, y_t\} \tag{7.4}$$

denote all measurements available up to time t. Before receiving the measurement y_t, the state distribution conditioned on the previous measurements is represented by

$$x_t \mid Y_{t-1} \sim \mathcal{N}\left(\hat{x}_{t|t-1}, P_{t|t-1}\right) \tag{7.5}$$

Thus, before y_t is observed, the best available state estimate is $\hat{x}_{t|t-1}$, with uncertainty represented by $P_{t|t-1}$.

To incorporate the new measurement, the joint distribution of the state and measurement is considered. Since the system and noise distributions are Gaussian, the variables x_t and y_t, conditioned on the previous measurements, are jointly Gaussian.

The required moments are first derived. The conditional mean of the state is

$$E[x_t \mid Y_{t-1}] = \hat{x}_{t|t-1} \tag{7.6}$$

The conditional mean of the measurement becomes

$$E[y_t \mid Y_{t-1}] = E[Cx_t + v_t \mid Y_{t-1}] \tag{7.7}$$
$$= CE[x_t \mid Y_{t-1}] \tag{7.8}$$
$$= C\hat{x}_{t|t-1} \tag{7.9}$$

The covariance between the measurement and the state is

$$\begin{aligned}\operatorname{Cov}(y_t, x_t \mid Y_{t-1}) &= \operatorname{Cov}(Cx_t + v_t, x_t \mid Y_{t-1}) && (7.10)\\ &= C \operatorname{Cov}(x_t, x_t \mid Y_{t-1}) && (7.11)\\ &= CP_{t|t-1} && (7.12)\end{aligned}$$

Similarly,

$$P_{xy} = P_{yx}^T = P_{t|t-1}C^T \tag{7.13}$$

The measurement covariance is

$$\begin{aligned}\operatorname{Cov}(y_t \mid Y_{t-1}) &= \operatorname{Cov}(Cx_t + v_t, Cx_t + v_t \mid Y_{t-1}) && (7.14)\\ &= CP_{t|t-1}C^T + R_v && (7.15)\end{aligned}$$

Therefore,

$$\begin{bmatrix} x_t \\ y_t \end{bmatrix} \middle| Y_{t-1} \sim \mathcal{N}\left(\begin{bmatrix} \hat{x}_{t|t-1} \\ C\hat{x}_{t|t-1} \end{bmatrix}, \begin{bmatrix} P_{t|t-1} & P_{t|t-1}C^T \\ CP_{t|t-1} & CP_{t|t-1}C^T + R_v \end{bmatrix} \right) \tag{7.16}$$

The measurement update can then be obtained using the projection theorem. For two jointly Gaussian variables,

$$Z = \begin{bmatrix} X \\ Y \end{bmatrix} \sim \mathcal{N}\left(\begin{bmatrix} m_x \\ m_y \end{bmatrix}, \begin{bmatrix} P_x & P_{xy} \\ P_{xy}^T & P_y \end{bmatrix} \right) \tag{7.17}$$

the conditional distribution is

$$X|Y \sim \mathcal{N}\left(m_x + P_{xy}P_y^{-1}(y - m_y),\ P_x - P_{xy}P_y^{-1}P_{xy}^T\right) \tag{7.18}$$

The projection theorem therefore provides the conditional mean and covariance of the state after the measurement has been observed. Applying it to the joint distribution of x_t and y_t gives the measurement-update equations

$$\boxed{\hat{x}_{t|t} = \hat{x}_{t|t-1} + P_{t|t-1}C^T \left(CP_{t|t-1}C^T + R_v\right)^{-1} \left(y_t - C\hat{x}_{t|t-1}\right)} \tag{7.19}$$

and

$$\boxed{P_{t|t} = P_{t|t-1} - P_{t|t-1}C^T \left(CP_{t|t-1}C^T + R_v\right)^{-1} CP_{t|t-1}} \tag{7.20}$$

After the measurement update, the state estimate is propagated to the next time step. From the system dynamics,

$$x_{t+1} = Ax_t + Bu_t + w_t, \tag{7.21}$$

the predicted state estimate becomes

$$\boxed{\hat{x}_{t+1|t} = A\hat{x}_{t|t} + Bu_t} \tag{7.22}$$

The corresponding covariance is

$$P_{t+1|t} = \mathrm{Cov}(Ax_t + Bu_t + w_t \mid Y_t), \tag{7.23}$$

which gives

$$\boxed{P_{t+1|t} = AP_{t|t}A^T + R_w} \tag{7.24}$$

The predicted quantities at time $t + 1$ then become the prior quantities for the next iteration, and the procedure is repeated when the next measurement is received.

7.1.1 Overview

The Kalman filter therefore consists of two main stages: the measurement update and the time update.

Starting with the prior state estimate

$$\hat{x}_{k|k-1}, \qquad P_{k|k-1} \tag{7.25}$$

the current measurement y_k is first used to obtain the posterior state estimate

$$\hat{x}_{k|k} = \hat{x}_{k|k-1} + P_{k|k-1}C^T \left(CP_{k|k-1}C^T + R_v\right)^{-1} \left(y_k - C\hat{x}_{k|k-1}\right) \tag{7.26}$$

with covariance

$$P_{k|k} = P_{k|k-1} - P_{k|k-1}C^T \left(CP_{k|k-1}C^T + R_v\right)^{-1} CP_{k|k-1} \tag{7.27}$$

This corresponds to the measurement update.

The resulting posterior state estimate is then propagated forward through the system model according to

$$\hat{x}_{k+1|k} = A\hat{x}_{k|k} + Bu_k \tag{7.28}$$

and

$$P_{k+1|k} = AP_{k|k}A^T + R_w \tag{7.29}$$

This corresponds to the time update. The predicted state $\hat{x}_{k+1|k}$ then becomes the prior estimate for the next measurement step $\hat{x}_{k|k-1}$, and the same procedure is repeated.

7.2 Results

The Kalman filter was evaluated using the stochastic bilinear system employed in the previous chapter. The system matrices were

$$A(v) = \begin{bmatrix} 0.90 & v \\ 0.00 & 0.50 \end{bmatrix}, \qquad B(v) = \begin{bmatrix} 0 \\ 1 \end{bmatrix} \tag{7.30}$$

with

$$C = \begin{bmatrix} 1 & 0 \end{bmatrix} \tag{7.31}$$

The process and measurement noise covariance matrices were selected as

$$R_w = \begin{bmatrix} 0.05 & 0 \\ 0 & 0.05 \end{bmatrix}, \qquad R_v = \begin{bmatrix} 0.005 \end{bmatrix} \tag{7.32}$$

The initial state distribution was defined using

$$\mu_0 = \begin{bmatrix} 0 \\ 0 \end{bmatrix}, \qquad P_0 = \begin{bmatrix} 0.01 & 0 \\ 0 & 0.01 \end{bmatrix} \tag{7.33}$$

The controller weighting matrices were selected as

$$Q = 1, \qquad R_U = 0.1, \qquad R_V = 0.1 \tag{7.34}$$

and the optimization variables were constrained by

$$-5 \leq u_k \leq 5, \qquad -5 \leq v_k \leq 5 \tag{7.35}$$

The reference levels were 3.1 and -1.3, with the reference changing after 15 time steps.

7.2.1 Open-Loop

For the open-loop evaluation, the stochastic bilinear system was simulated over a horizon of 30 steps using 100 Monte Carlo realizations. The same optimized U and V sequences were used for all Monte Carlo realizations. The Kalman filter was then applied to the resulting noisy measurements.

7.2.1.1 Single Trajectory

A single Monte Carlo realization was first selected to illustrate the estimation behavior of the Kalman filter. The estimated states were compared directly with the corresponding true states.

The uncertainty associated with the state estimate is also shown using the $\pm 2\sigma$ interval obtained from the posterior covariance matrix. This provides a visual indication of the uncertainty represented by the filter.

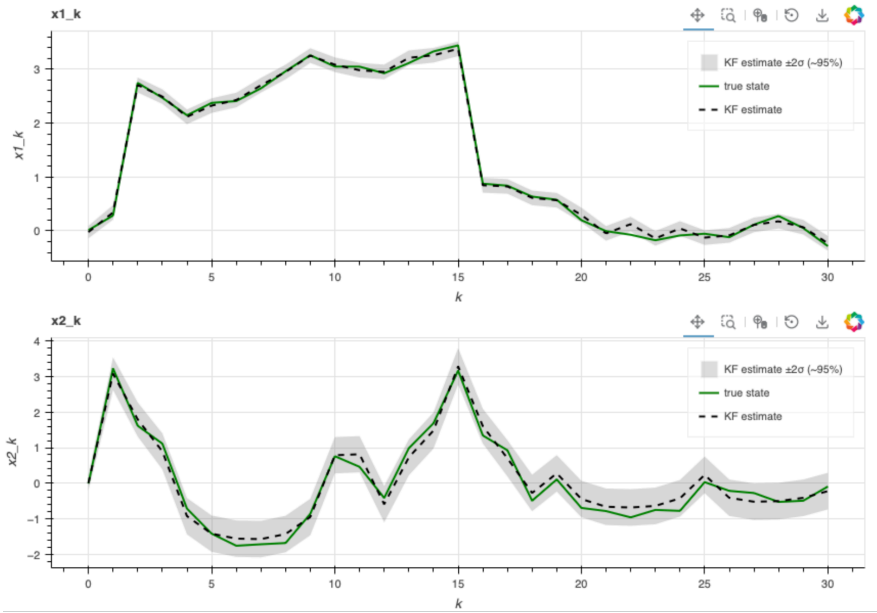


Figure 7.1: Kalman-filter state estimate for a single stochastic bilinear open-loop trajectory.

7.2.1.2 Monte Carlo Result

The Kalman filter was then applied to all Monte Carlo trajectories. The true state trajectories and their corresponding Kalman-filter estimates are shown together in order to examine the estimation behavior over repeated stochastic realizations.

The empirical mean of the true trajectories and the empirical mean of the estimated trajectories are also displayed. In addition, the corresponding $\pm 2\sigma$ regions show the spread of the Monte Carlo trajectories.

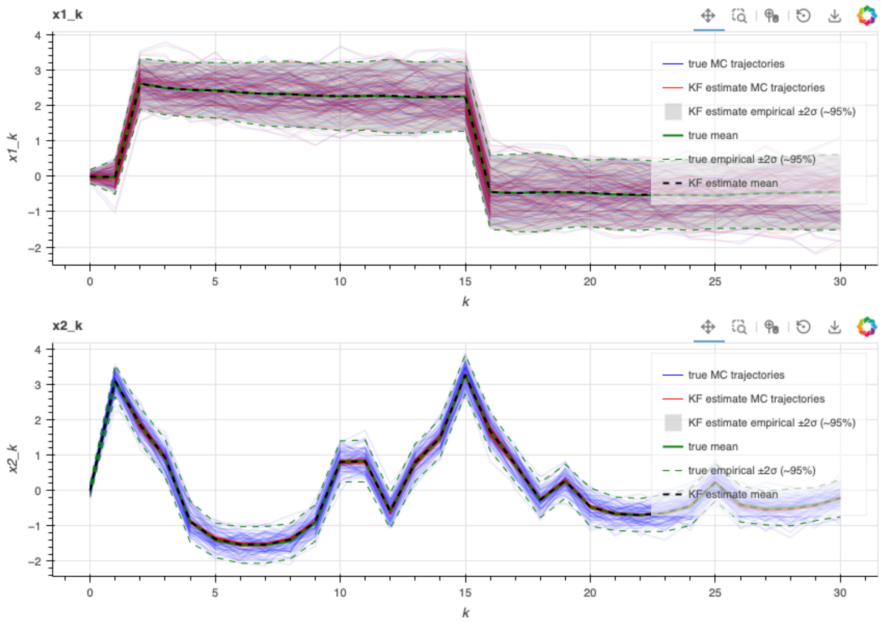


Figure 7.2: Monte Carlo true-state trajectories and Kalman-filter estimates for the stochastic bilinear open-loop simulation.

7.2.1.3 Performance Evaluation

The performance of the Kalman filter was further evaluated by examining the difference between the estimated and true states over all Monte Carlo realizations.

The mean estimation error is shown together with the empirical $\pm 2\sigma$ spread of the estimation error. This is compared with the uncertainty predicted by the Kalman covariance matrix.

This comparison provides an indication of whether the covariance calculated by the Kalman filter appropriately represents the observed estimation uncertainty over the Monte Carlo simulations.

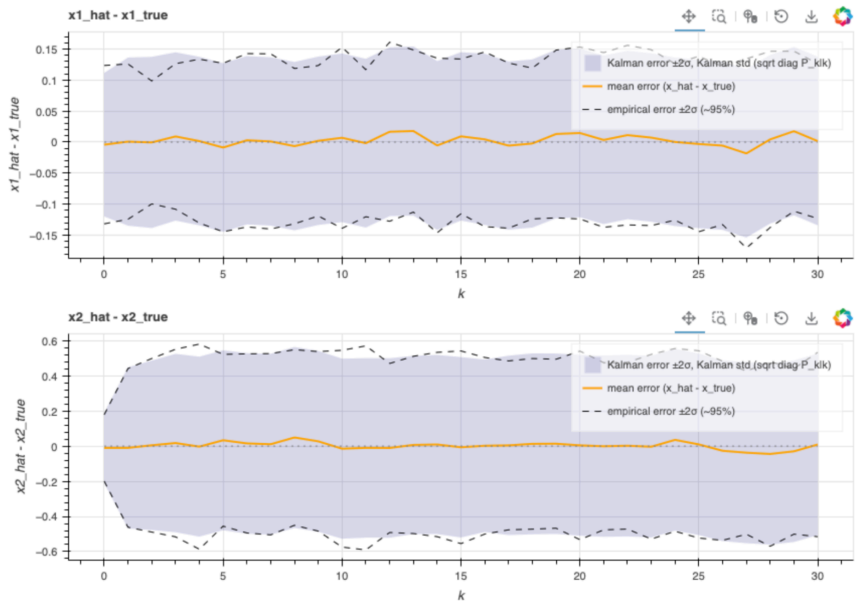


Figure 7.3: Mean estimation error and comparison between the empirical and Kalman-filter uncertainty for the stochastic bilinear open-loop simulation.

7.2.2 Closed-Loop

The Kalman filter was also evaluated using the stochastic bilinear closed-loop simulations. The same system and stochastic parameters were used.

In this case, the input sequences U and V can differ between Monte Carlo realizations. Consequently, the corresponding time-varying system matrices used in the Kalman-filter prediction step also depend on the individual realization.

7.2.2.1 Single Trajectory

A single closed-loop Monte Carlo realization was first selected. The Kalman-filter estimate was obtained using the U and V sequences corresponding to that particular trajectory.

The true states, estimated states, and the corresponding $\pm 2\sigma$ uncertainty interval are shown in Figure 7.4.

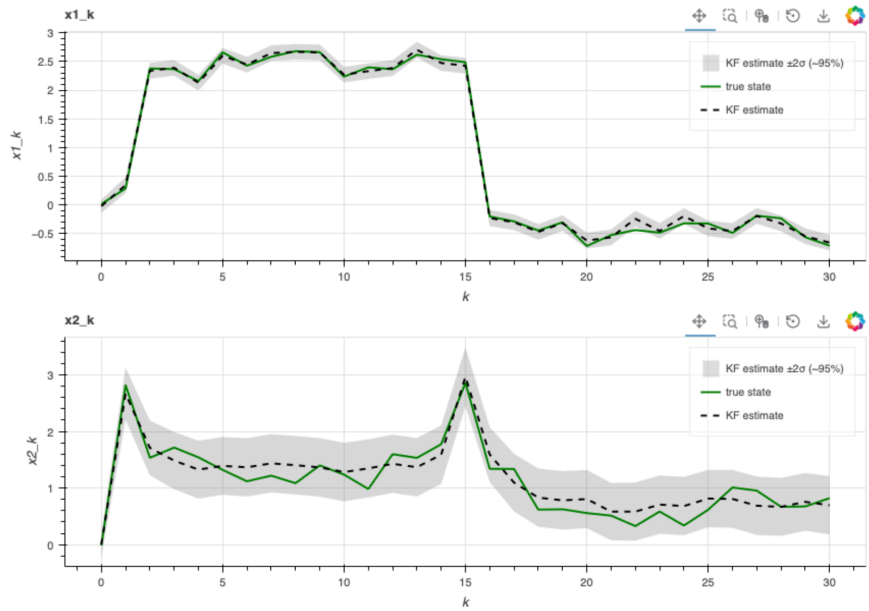


Figure 7.4: Kalman-filter state estimate for a single stochastic bilinear closed-loop trajectory.

7.2.2.2 Monte Carlo Result

The filter was subsequently applied to every closed-loop Monte Carlo trajectory. Since the optimized V-sequence can differ between realizations, the corresponding covariance propagation is also evaluated separately for each trajectory.

The true and estimated state trajectories are shown together with their empirical means and spreads.

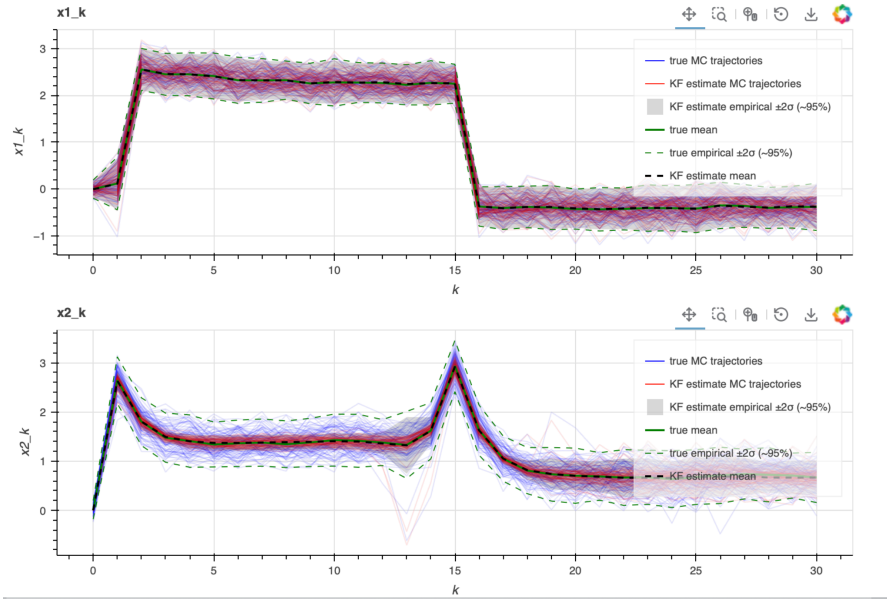


Figure 7.5: Monte Carlo true-state trajectories and Kalman-filter estimates for the stochastic bilinear closed-loop simulation.

7.2.2.3 Performance Evaluation

Finally, the estimation errors from all closed-loop Monte Carlo trajectories were analyzed.

The mean estimation error and its empirical spread are compared with the uncertainty obtained from the Kalman covariance matrices. Since each trajectory may have a different V-sequence, the covariance evolution can also differ between simulations.

The comparison therefore provides a statistical evaluation of the state estimator under the stochastic bilinear closed-loop simulations.

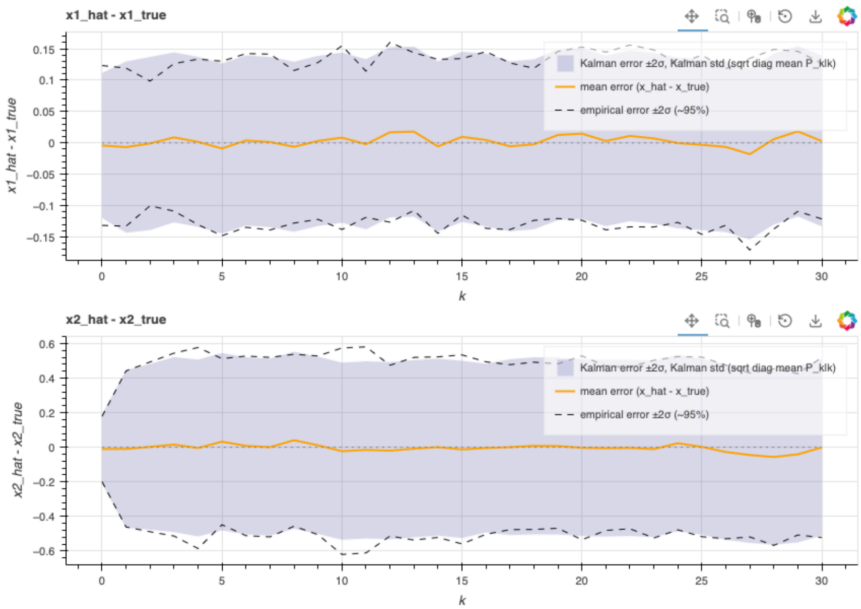


Figure 7.6: Mean estimation error and comparison between the empirical and Kalman-filter uncertainty for the stochastic bilinear closed-loop simulation.

CHAPTER 8

District Heating System

8.1 Derivation of $\dot{T}_i$ Equation

The district heating pipes are assumed to transport incompressible water with constant specific heat capacity. The flow velocity is assumed to be constant throughout each pipe, and only temperature variations in the axial direction are considered.

For a pipe with axial coordinate z, the energy balance is written as

$$\partial_t u(t,z) = -\partial_z \mathcal{N}_h(t,z) + q(t,z), \qquad z \in [z_0, z_f] \tag{8.1}$$

where $u(t,z)$ denotes the internal energy per unit volume, $\mathcal{N}_h(t,z)$ denotes the enthalpy flux, and $q(t,z)$ represents the heat loss per unit volume.

The enthalpy flux is expressed as

$$\mathcal{N}_h(t,z) = H\left(T(t,z), p, \mathcal{N}(t)\right) \tag{8.2}$$

where $T(t,z)$ is the water temperature, p is the pressure, and $\mathcal{N}(t)$ is the molar flux. The molar flux is given by

$$\mathcal{N}(t) = \rho v(t) \tag{8.3}$$

where ρ denotes the constant density and $v(t)$ denotes the flow velocity.

The heat loss to the surroundings is modeled as

$$q(t,z) = -k\left(T(t,z) - T_a(t)\right) \tag{8.4}$$

where k is the conductance and $T_a(t)$ is the ambient temperature.

Applying the chain rule to the enthalpy flux gives

$$\partial_T H\left(T(t,z), p, \mathcal{N}(t)\right) \partial_z T(t,z) = \partial_z \mathcal{N}_h(t,z) \tag{8.5}$$

Using the specific heat capacity c_p, this becomes

$$\mathcal{N}(t) c_p \partial_z T(t,z) = \partial_z \mathcal{N}_h(t,z) \tag{8.6}$$

The pipe is then segmented into M finite volumes. For the i-th segment,

$$\Omega_i = [z_i, z_{i+1}] \tag{8.7}$$

and the energy balance is integrated over the segment:

$$\partial_t \int_{\Omega_i} u(t,z)\,dz = -\int_{\Omega_i} \partial_z \mathcal{N}_h(t,z)\,dz + \int_{\Omega_i} q(t,z)\,dz \tag{8.8}$$

Multiplying by the cross-sectional area A, the energy balance of the segment can be written as

$$\dot{U}_i(t) = -A\mathcal{N}(t)c_p \int_{z_i}^{z_{i+1}} \partial_z T(t,z)\,dz + Q_i(t) \tag{8.9}$$

where $U_i(t)$ denotes the internal energy of the segment and $Q_i(t)$ denotes the corresponding heat loss.

Evaluating the integral gives

$$\dot{U}_i(t) = -A\mathcal{N}(t)c_p \left(T(t,z_{i+1}) - T(t,z_i)\right) + Q_i(t) \tag{8.10}$$

Using the average temperature of each finite volume, this is approximated as

$$\dot{U}_i(t) = -A\mathcal{N}(t)c_p \left(\bar{T}_{i+1}(t) - \bar{T}_i(t)\right) + Q_i(t) \tag{8.11}$$

The internal energy of the i-th segment is written as

$$U_i(t) = nc_p\bar{T}_i(t) \tag{8.12}$$

where

$$n = A\Delta z\rho, \qquad \Delta z = z_{i+1} - z_i \tag{8.13}$$

The heat loss of the segment is approximated by

$$Q_i(t) \approx -A\Delta z k \left(\bar{T}_i(t) - T_a(t)\right) \tag{8.14}$$

Substituting the internal-energy and heat-loss expressions into the energy balance gives

$$\begin{aligned} c_p A\Delta z\rho\,\dot{\bar{T}}_i(t) = &-A\mathcal{N}(t)c_p \left(\bar{T}_{i+1}(t) - \bar{T}_i(t)\right) \\ &- A\Delta z k \left(\bar{T}_i(t) - T_a(t)\right) \end{aligned} \tag{8.15}$$

Since

$$\mathcal{N}(t) = \rho v(t) \tag{8.16}$$

the equation can be divided by $A\Delta z\rho c_p$, resulting in

$$\boxed{\dot{\bar{T}}_i(t) = -\frac{v(t)}{\Delta z}\left(\bar{T}_{i+1}(t) - \bar{T}_i(t)\right) - \frac{k}{\rho c_p}\left(\bar{T}_i(t) - T_a(t)\right)} \tag{8.17}$$

The resulting pipe model can subsequently be expressed in continuous state-space form as

$$\dot{x}(t) = A_c x(t) + B_c u(t) + E_c d(t) \tag{8.18}$$
$$y(t) = C_c x(t) + e(t_k) \tag{8.19}$$

where the states $x(t)$ correspond to the average temperatures $\bar{T}_i(t)$, $u(t)$ represents the supply temperature T_s, $d(t)$ represents the ambient temperature T_a, and $y(t)$ contains the temperatures measured at the selected pipe nodes.

8.2 Change of Variable

From the temperature equation derived in the previous section,

$$\dot{T}_i = -\frac{v}{\Delta z}\left(T_i - T_{i-1}\right) - \frac{k}{\rho c_p}\left(T_i - T_a\right) \tag{8.20}$$

To simplify the notation, define

$$\alpha(v) = \frac{v}{\Delta z} \tag{8.21}$$

and

$$\beta = \frac{k}{\rho c_p} \tag{8.22}$$

The temperature dynamics can then be written as

$$\dot{T}_i = -\alpha(v)\left(T_i - T_{i-1}\right) - \beta\left(T_i - T_a\right) \tag{8.23}$$

A change of variables is introduced by defining the state relative to the ambient temperature as

$$x_i = T_i - T_a \tag{8.24}$$

Similarly, the input is defined as the supply temperature relative to the ambient temperature,

$$u = T_s - T_a \tag{8.25}$$

Therefore,

$$T_i = x_i + T_a \tag{8.26}$$

and

$$T_s = u + T_a \tag{8.27}$$

Differentiating (8.24) gives

$$\dot{x}_i = \dot{T}_i - \dot{T}_a \tag{8.28}$$

Substituting

$$T_i = x_i + T_a, \qquad T_{i-1} = x_{i-1} + T_a \tag{8.29}$$

into (8.23) gives

$$\dot{x}_i + \dot{T}_a = -\alpha \left[(x_i + T_a) - (x_{i-1} + T_a) \right] - \beta x_i \tag{8.30}$$

After simplifying,

$$\boxed{\dot{x}_i = -(\alpha + \beta) x_i + \alpha x_{i-1} - \dot{T}_a} \tag{8.31}$$

For the first pipe segment, the upstream temperature is the supply temperature T_s. Since

$$u = T_s - T_a \tag{8.32}$$

the first state equation becomes

$$\boxed{\dot{x}_0 = -(\alpha + \beta) x_0 + \alpha u - \dot{T}_a} \tag{8.33}$$

Collecting all pipe-segment temperatures into the state vector gives the continuous-time system

$$\boxed{\dot{x} = A_c(v)x + B_c(v)u + E_c\dot{T}_a} \tag{8.34}$$

The system matrix is

$$A_c(v) = \begin{bmatrix} -(\alpha+\beta) & 0 & 0 & \cdots & 0 \\ \alpha & -(\alpha+\beta) & 0 & \cdots & 0 \\ 0 & \alpha & -(\alpha+\beta) & \ddots & \vdots \\ \vdots & \ddots & \ddots & \ddots & 0 \\ 0 & \cdots & 0 & \alpha & -(\alpha+\beta) \end{bmatrix} \tag{8.35}$$

where

$$\alpha = \alpha(v) = \frac{v}{\Delta z} \tag{8.36}$$

The input matrix is

$$B_c(v) = \begin{bmatrix} \alpha \\ 0 \\ \vdots \\ 0 \end{bmatrix} \tag{8.37}$$

and the disturbance matrix associated with the change in ambient temperature is

$$E_c = \begin{bmatrix} -1 \\ -1 \\ \vdots \\ -1 \end{bmatrix} \tag{8.38}$$

8.3 Controller and Hidden Plant Approach

After introducing the change of variable

$$x_i = T_i - T_a \tag{8.39}$$

the continuous-time district heating model was obtained as

$$\dot{x} = A_c(v)x + B_c(v)u + E_c\dot{T}_a \tag{8.40}$$

The change of variable therefore introduces the derivative of the ambient temperature into the state dynamics. In the implementation, the model used by the controller and the model used to simulate the physical plant are treated separately.

8.3.1 Controller Approach

On the controller side, the ambient temperature is assumed to remain constant within each sampling interval. Thus,

$$T_a(t) = T_{a,k}, \qquad t \in [t_k, t_{k+1}) \tag{8.41}$$

and consequently

$$\dot{T}_a(t) = 0, \qquad t \in (t_k, t_{k+1}) \tag{8.42}$$

Therefore,

$$E_c\dot{T}_a = 0 \tag{8.43}$$

and the continuous-time model used by the controller reduces to

$$\boxed{\dot{x} = A_c(v)x + B_c(v)u} \tag{8.44}$$

This model is discretized using a Zero-Order Hold assumption. During each sampling interval, the matrices and the input are kept constant. The resulting state at the end of the sampling interval is

$$x(t_{k+1}^-) = e^{A_{c,k}\Delta t} x_k + \int_0^{\Delta t} e^{A_{c,k}(\Delta t - \tau)} B_{c,k} u_k \, d\tau \tag{8.45}$$

The discrete-time matrices are obtained from

$$\exp\left(\begin{bmatrix} A_c & B_c \\ 0 & 0 \end{bmatrix} \Delta t\right) = \begin{bmatrix} A_k & B_k \\ 0 & I \end{bmatrix} \tag{8.46}$$

Thus, the controller uses the discrete-time model generated from the continuous dynamics under the assumption that the ambient temperature does not change inside the sampling interval.

8.3.2 Hidden Plant Approach

The hidden plant represents the system used to generate the simulated measurements. In contrast to the controller model, the hidden plant is affected by the actual ambient-temperature values.

Because the transformed state is defined relative to the ambient temperature, a change in T_a between two sampling instants changes the state coordinates. After the continuous evolution over one sampling interval,

$$x_{k+1} = x_{k+1}^- + (T_{a,k} - T_{a,k+1})\,\mathbf{1} \tag{8.47}$$

The ambient-temperature contribution is therefore defined as

$$d_k = (T_{a,k} - T_{a,k+1})\,\mathbf{1} \tag{8.48}$$

The hidden plant simulation consequently uses

$$\boxed{x_{k+1} = A_k x_k + B_k u_k + d_k + w_k} \tag{8.49}$$

where

$$w_k \sim \mathcal{N}(0, R_w) \tag{8.50}$$

The output represents the outlet temperature relative to the current ambient temperature,

$$y_k = T_{\mathrm{out},k} - T_{a,k} \tag{8.51}$$

The measured output is modeled as

$$\boxed{y_k = C x_k + \nu_k} \tag{8.52}$$

with

$$\nu_k \sim \mathcal{N}(0, R_v) \tag{8.53}$$

The hidden plant is used only to simulate the underlying system behavior. Its internal state is not assumed to be directly available to the controller. Instead, the controller receives the noisy output measurements, and the Kalman filter is applied to these measurements in order to estimate the internal state.

Therefore, the controller performs its prediction using the model in which the ambient temperature is assumed constant over each sampling interval, whereas the hidden plant evolves according to the actual ambient-temperature sequence through the additional term d_k. This separation allows the effect of changing ambient conditions to be included in the simulation while maintaining the controller formulation based on the ZOH-discretized model.

8.3.3 Change on Kalman Filter

The change of variable

$$x_i = T_i - T_a \tag{8.54}$$

also introduces a modification to the Kalman-filter procedure. Since the state is defined relative to the ambient temperature, a change in the ambient temperature between two consecutive sampling instants changes the state coordinates.

As derived previously,

$$x_{k+1} = x_{k+1}^{-} + (T_{a,k} - T_{a,k+1})\,\mathbf{1} \tag{8.55}$$

Defining

$$d_k = (T_{a,k} - T_{a,k+1})\,\mathbf{1} \tag{8.56}$$

the same correction must also be applied to the predicted state estimate of the Kalman filter.

The measurement-update equations remain unchanged. Starting from the prior estimate $\hat{x}_{k|k-1}$, the measurement y_k is used to obtain

$$\hat{x}_{k|k} = \hat{x}_{k|k-1} + P_{k|k-1}C^T \left(CP_{k|k-1}C^T + R_v\right)^{-1} \left(y_k - C\hat{x}_{k|k-1}\right) \tag{8.57}$$

and

$$P_{k|k} = P_{k|k-1} - P_{k|k-1}C^T \left(CP_{k|k-1}C^T + R_v\right)^{-1} CP_{k|k-1} \tag{8.58}$$

The time update is then first performed using the controller model,

$$\hat{x}_{k+1|k}^{-} = A_k\hat{x}_{k|k} + B_k u_k \tag{8.59}$$

At this stage, the predicted state is based on the assumption used by the controller that the ambient temperature remains constant over the sampling interval.

When the next ambient-temperature value $T_{a,k+1}$ becomes available, the change in ambient temperature is calculated through

$$d_k = (T_{a,k} - T_{a,k+1})\,\mathbf{1} \tag{8.60}$$

The predicted state estimate is then corrected as

$$\boxed{\hat{x}_{k+1|k} = \hat{x}^{-}_{k+1|k} + d_k} \tag{8.61}$$

Therefore, the complete prediction step can be written as

$$\boxed{\hat{x}_{k+1|k} = A_k \hat{x}_{k|k} + B_k u_k + d_k} \tag{8.62}$$

After this correction, the time index is advanced,

$$k \leftarrow k + 1 \tag{8.63}$$

and the corrected prediction becomes the prior estimate used in the next measurement update.

Thus, the main modification to the Kalman-filter structure is the additional state correction introduced when the new ambient-temperature value becomes available. This allows the state estimate to remain consistent with the definition $x_i = T_i - T_a$ while the actual ambient temperature changes over time.

8.4 Updates on Implementation

Two modifications were introduced in the implementation of the optimization problem. First, the changes in the control input U and flow velocity V were included in the cost function. Second, the numerical optimization method was changed from SLSQP to IPOPT due to the increase in computational time.

8.4.1 Penalizing Changes in U and V

Previously, the objective function penalized the tracking error together with the magnitudes of the control input U and the flow velocity V. However, minimizing only these quantities does not explicitly discourage large changes between consecutive control actions.

Therefore, additional terms were introduced to penalize the changes in U and V. For consecutive prediction steps, these changes are defined as

$$\Delta u_k = u_k - u_{k-1} \tag{8.64}$$

and

$$\Delta v_k = v_k - v_{k-1} \tag{8.65}$$

The corresponding penalties are

$$\Delta u_k^T R_{\Delta U} \Delta u_k \tag{8.66}$$

and

$$\Delta v_k^T R_{\Delta V} \Delta v_k \tag{8.67}$$

The updated objective function therefore becomes

$$\begin{aligned}\phi(U,V) = \frac{1}{2}\sum_{k=0}^{N-1} \Big[&(\mu_{y,k} - \bar{y}_k)^T Q(\mu_{y,k} - \bar{y}_k) + u_k^T R_U u_k + v_k^T R_V v_k \\ &+ \operatorname{tr}(QP_{y,k}) + \Delta u_k^T R_{\Delta U} \Delta u_k + \Delta v_k^T R_{\Delta V} \Delta v_k \Big]\end{aligned} \tag{8.68}$$

Thus, R and S penalize the magnitude of the control variables, whereas Q_{change} and S_{change} penalize how rapidly these variables change from one time step to the next.

For $k > 0$,

$$\Delta u_k = u_k - u_{k-1}, \tag{8.69}$$
$$\Delta v_k = v_k - v_{k-1} \tag{8.70}$$

In the receding-horizon implementation, the first elements of the newly optimized sequences are compared with the values that were actually applied at the previous sampling instant. Therefore,

$$\Delta u_0 = u_0 - u_{\text{prev}}, \tag{8.71}$$
$$\Delta v_0 = v_0 - v_{\text{prev}} \tag{8.72}$$

When no previous control value exists, such as at the beginning of the simulation, this first change term is not penalized.

Including these additional terms encourages smoother control-input and flow-velocity sequences and avoids unnecessarily large changes between consecutive optimization steps.

8.4.2 Numerical Solver Change

The numerical solver used for the nonlinear optimization problem was also changed during the implementation.

The earlier stochastic bilinear models were solved using Sequential Least Squares Programming (SLSQP). As the district heating model was extended, the optimization problem became more computationally demanding. In particular, the larger state dimension, propagation of the stochastic covariance, chance constraints, bilinear dependence on V, and the additional penalties on the changes in U and V increased the computation time.

For this reason, the implementation was changed from SLSQP to IPOPT. The optimization problem is constructed using CasADi.

8.5 Results

The developed controller was evaluated on the single-pipe district heating example. The pipe was divided into $M = 6$ finite volumes over a total length of 6 km, and the sampling time was selected as $\Delta t = 0.5\,\mathrm{h}$. The heat-loss coefficient was

$$\beta = 7.66 \times 10^{-3}\,\mathrm{h}^{-1} \tag{8.73}$$

Two temperatures were considered as measured outputs: the temperature at the pipe outlet and the temperature at the third finite volume. The initial temperature of the pipe was 65°C, while the initial ambient temperature was 0°C.

The physical output constraints were selected as

$$65^\circ\mathrm{C} \leq T_{\mathrm{out}} \leq 90^\circ\mathrm{C} \tag{8.74}$$

for both measured locations. The supply-temperature and flow-velocity bounds were

$$50^\circ\mathrm{C} \leq T_s \leq 110^\circ\mathrm{C} \tag{8.75}$$

and

$$0.05 \leq v \leq 7.0 \quad \mathrm{km/h} \tag{8.76}$$

The reference temperatures were changed during the simulation according to

$$\bar{T}_1 = \begin{bmatrix} 70 & 70 \end{bmatrix}^T, \qquad \bar{T}_2 = \begin{bmatrix} 70 & 87 \end{bmatrix}^T, \qquad \bar{T}_3 = \begin{bmatrix} 87 & 70 \end{bmatrix}^T \tag{8.77}$$

Each reference level was held for 48 sampling instants, corresponding to 24 h. The ambient temperature was initially 0°C and was changed to 10°C at $t = 36\,\mathrm{h}$.

The weighting matrices used in the optimization were

$$Q = 10^{11} I, \qquad R_U = 0, \qquad R_V = 0 \tag{8.78}$$

together with the move-suppression weights

$$R_{\Delta U} = 10, \qquad R_{\Delta V} = 20 \tag{8.79}$$

The stochastic behavior was evaluated using 200 Monte Carlo realizations.

8.5.1 Open-Loop

In the open-loop simulation, the optimization problem was solved once over the complete simulation horizon. The complete future ambient-temperature profile was provided to the controller before the optimization. Therefore, the calculated sequences U^* and V^* already account for the scheduled change in ambient temperature.

The same optimized input and velocity sequences were subsequently applied to all Monte Carlo realizations.

8.5.2 Closed-Loop

For the closed-loop simulation, a prediction horizon of

$$N = 12 \tag{8.80}$$

was used, corresponding to 6 h. In contrast to the open-loop case, the controller does not receive the future ambient-temperature trajectory. At each sampling instant, only the currently available ambient temperature is used and is assumed to remain constant over the prediction horizon.

A separate MPC optimization is performed for every Monte Carlo realization. Consequently, each trajectory obtains its own optimal U^* and V^* sequences according to its current state estimate. Only the first elements of these sequences are applied, after which a new measurement is received and the optimization is repeated.

When a new ambient-temperature value becomes available, the Kalman-filter state prediction is corrected using the ambient-temperature re-basing term. The corrected state estimate is then used by the subsequent MPC problem. This allows the controller to respond to the actual ambient-temperature change without assuming its future value beforehand.

CHAPTER 9

Conclusion

During this internship, a complete MPC framework was developed step by step from linear deterministic systems to stochastic bilinear systems with chance constraints. The work also incorporated state-input analysis and Kalman filtering for state estimation under noisy measurements. The resulting controller was applied to a district heating system and evaluated in both open-loop and closed-loop settings using Monte Carlo simulations. The final implementation combines nonlinear optimization, uncertainty handling, probabilistic constraints, and state estimation, providing a basis for further development of stochastic MPC methods for district heating applications

APPENDIX A

An Appendix

Codes used in this report can be found on: